Theoretical Framework: Generative Surrogates for PDE Solution Fields

Project: laplace-uq-bench **Author:** Jane Yeung **Date:** March 2026 **Companion to:** `laplace-uq-bench/` implementation plan **Python package:** `diffphys` (code paths use `src/diffphys/` throughout; the repository was renamed but the Python package name is retained for import stability)

Table of Contents

1. Motivation & Physical Setup
2. The Forward Problem: Laplace's Equation
3. Neural Surrogate Architectures
4. Score-Based Diffusion and Langevin Dynamics
5. Posterior Inference Under Uncertain Observations
6. Physics-Informed Regularization
7. Conditional Flow Matching and Optimal Transport
8. Improved DDPM Training
9. Conformal Prediction for Calibrated Uncertainty
10. Diffusion Posterior Sampling
11. Evaluation Theory: Proper Scoring and Calibration
12. Reference Paper Connections
13. Theory-to-Deliverable Mapping
14. Validation Criteria
15. Notation Reference
16. Bibliography

---

1. Motivation & Physical Setup

1.1 The Physical System

We study the **steady-state heat equation** (Laplace's equation) on the unit square $\Omega = [0,1]^2$ with Dirichlet boundary conditions:

```
\nabla^2 T(x,y) = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \qu
```

```
T = g(x,y) \quad \text{on } \partial\Omega \tag{1.2}
```

where $T(x,y)$ is the temperature field and $g(x,y)$ prescribes the boundary profile. Laplace's equation arises as the equilibrium limit of the heat equation $\partial_t T = \kappa \nabla^2 T$ — when all transient behavior has decayed, the temperature distribution satisfies (1.1). It is the prototypical second-order elliptic PDE: linear, self-adjoint, and admits a unique classical solution for any continuous Dirichlet data g on a bounded domain with smooth boundary.

1.2 Why This System

Laplace's equation is chosen *precisely because it is simple.* The PDE has a known, fast numerical solver — a finite-difference discretization with LU factorization solves each instance in $O(1)$ ms.

Neural surrogates cannot beat this solver on accuracy or speed for this problem.

The simplicity is the point. Because the numerical oracle is cheap and its truncation error is negligible for smooth BCs at $N = 64$ (see §2.2), we can: 1. Generate arbitrarily large datasets with negligible solver error. 2. Evaluate neural surrogates against an oracle whose solutions are accurate to well below the resolution any neural surrogate will achieve. 3. Isolate the question "when does a generative model add value?" from confounding factors like solver error, mesh dependence, or turbulence modeling.

This is a **controlled experimental setting** for studying neural surrogate uncertainty, not a claim that diffusion models should replace PDE solvers.

1.3 Why These Computational Methods

The project proceeds in two phases. **Phase 1** establishes deterministic baselines and the DDPM generative framework; **Phase 2** compares three approaches to uncertainty quantification under noisy boundary observations, with a fourth as a stretch goal. Sections 1–6 cover the PDE setup, baseline architectures, DDPM theory, and evaluation foundations shared by both phases; §7–§10 extend the project with the Phase 2 methods: flow matching, improved DDPM training, conformal prediction, and DPS.

Method	Type	Uncertainty?	Why included	Phase
U-Net regressor	Deterministic	No	Simplest deep learning baseline	1
FNO	Deterministic	No	Operator-learning inductive bias (underperformed; see §3.2)	1
Deep ensemble (5×)	Probabilistic, non-generative	Epistemic only	Calibration baseline	1–2
Ensemble + conformal	Probabilistic, non-generative	Post-hoc calibration wrapper	Coverage calibration at zero training cost	2
Conditional DDPM (improved)	Probabilistic, generative	Approximate posterior samples	Best functional CRPS on this benchmark (sparse-noisy, matched 5v5)	2
Conditional flow matching	Probabilistic, generative	Approximate posterior samples	Alternative generative framework	2
DPS (unconditional + guidance)	Probabilistic, generative	Approximate posterior samples	Stretch goal: zero-shot adaptation	2

The Phase 2 comparison tests three UQ approaches: **(1)** improved DDPM (§8) with cosine schedule, v-prediction, and Min-SNR weighting, which achieved the best functional CRPS on this benchmark (sparse-noisy regime, matched 5v5) and tightest pixelwise conformal intervals; **(2)** conditional flow matching (§7), which offered simpler training dynamics but underperformed DDPM on most metrics

in this setting — suggesting that on this problem, the training improvements (§8) were more impactful than the choice of generative framework; and **(3)** deep ensembles with pixelwise conformal prediction (§9), a practical post-hoc calibration wrapper; the exact finite-sample coverage guarantee applies to standard split conformal prediction (§9.1, Eq. 9.4), while the pooled-pixel variant used in the benchmark is an empirical adaptation (see §9.1 exchangeability caveat). DPS (§10) is a stretch goal exploring inference-time adaptation.

All models are implemented in `src/diffphys/model/`*. The U-Net backbone (*`model/unet.py`*) is shared across the regressor, ensemble members, DDPM, and flow matching. The FNO (*`model/fno.py`*) uses independent architecture. All conditional models consume the 8-channel conditioning tensor defined in §2.5 and produced by* `src/diffphys/data/conditioning.py`*.*

---

2. The Forward Problem: Laplace's Equation

2.1 Existence, Uniqueness, and Maximum Principle

Theorem (Existence and Uniqueness). For continuous Dirichlet data $g \in C(\partial\Omega)$ on a bounded domain $\Omega \subset \mathbb{R}^2$ with smooth boundary, there exists a unique solution $T \in C^2(\Omega) \cap C(\bar{\Omega})$ to (1.1)–(1.2). [Exact; see Evans, *Partial Differential Equations*, 2nd ed., §2.2.3 (uniqueness via the maximum principle).]

Domain note. The unit square $\Omega = [0,1]^2$ has corners, so "smooth boundary" is technically too strong. The existence and uniqueness result extends to Lipschitz domains (which the square is) via weak solution theory (Evans, Ch. 6), and the solution is C^∞ in the interior. Corner singularities in the gradient can arise if the boundary data has discontinuities at corners, but our corner-consistent BC generation (§2.4) ensures continuous boundary data, so the classical solution exists and the maximum principle applies.

Theorem (Strong Maximum Principle). If T is harmonic on Ω (i.e., $\nabla^2 T = 0$) and not identically constant, then T attains its maximum and minimum on $\partial\Omega$ only. [Exact; Evans, 2nd ed., §2.2.3.]

Consequence for evaluation. The maximum principle provides a physics-based test: for any neural surrogate prediction $\hat{T}$, every interior point must satisfy

```
\min_{\partial\Omega} g \leq \hat{T}(x,y) \leq \max_{\partial\Omega} g \quad \forall (x,y) \in
```

Violations of (2.1) are unphysical and indicate that the surrogate has not learned the elliptic structure. This is implemented in `src/diffphys/evaluation/physics.py:check_maximum_principle()`.

2.2 Finite-Difference Discretization

We discretize Ω on a uniform grid with spacing $h = 1/(N-1)$, where $N = 64$. Grid points are indexed as $(x_i, y_j) = (ih, jh)$ for $i, j = 0, \dots, N-1$.

Array indexing convention. Throughout the repository, arrays use the convention $T[i,j] = T(y_i, x_j)$, with axis 0 corresponding to the vertical (y) direction and axis 1 to the horizontal (x) direction. Top-edge values occupy row $i = N-1$; bottom-edge values occupy row $i = 0$; left-edge values occupy column $j = 0$; right-edge values occupy column $j = N-1$.

The 5-point stencil. At each interior point (i, j) with $1 \leq i, j \leq N-2$, the Laplacian is approximated by the standard second-order central difference (LeVeque, 2007, Ch. 3):

```
\nabla^2 T \approx \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j}}{h^2} = 0. \
```

This yields the linear system:

```
L \mathbf{u} = \mathbf{b} \tag{2.3}
```

where $\mathbf{u} \in \mathbb{R}^{(N-2)^2}$ contains the interior unknowns (row-major ordering), L is the sparse $(N-2)^2 \times (N-2)^2$ Laplacian matrix, and $\mathbf{b}$ absorbs the boundary values.

Construction of L. [Exact.] Using row-major indexing $k = i \cdot (N-2) + j$ for the interior grid $(i, j) \in \{0, \dots, N-3\}^2$, the Laplacian matrix has the block tridiagonal structure:

```
L = \frac{1}{h^2} \begin{pmatrix} B_L & I & & \\ I & B_L & I & \\ & \ddots & \ddots & \ddots \\
```

where B_L is the $(N-2) \times (N-2)$ tridiagonal matrix

```
B_L = \begin{pmatrix} -4 & 1 & & \\ 1 & -4 & 1 & \\ & \ddots & \ddots & \ddots \\ & & 1 & -4 \
```

and I is the $(N-2) \times (N-2)$ identity matrix.

Construction of b. The right-hand side vector absorbs boundary values. For interior point (i, j), any neighbor that lies on $\partial\Omega$ contributes $-g_{\text{boundary}}/h^2$ to the corresponding entry of $\mathbf{b}$. Corner and edge-adjacent points pick up contributions from 1–2 boundary neighbors. Interior points far from the boundary have $b_k = 0$.

This is implemented in `src/diffphys/pde/laplace.py:build_laplacian_matrix()` and `assemble_rhs()`. The h^2 factor is absorbed into the matrix for numerical convenience.

LU factorization and reuse. [Exact.] Since L depends only on the grid size N and not on the boundary data g, we compute the sparse LU factorization $L = P^{-1}L'U'$ once using `scipy.sparse.linalg.splu`. Each subsequent solve requires only forward/backward substitution:

```
\mathbf{u} = L^{-1} \mathbf{b} \quad \text{via } \text{lu\_factor.solve(b)} \tag{2.6}
```

Cost per solve: $O((N-2)^2)$ flops for back-substitution. On a modern laptop CPU, each solve takes on the order of 0.5ms for $N = 64$; generating 50,000 solutions takes on the order of 25 seconds. (Exact timings are hardware-dependent.)

The solver with LU reuse is implemented in `src/diffphys/pde/laplace.py:LaplaceSolver`. The factorization is computed once in the constructor. Dataset generation uses `pde/generate.py`.

Truncation error and numerical oracle convention. The 5-point stencil has truncation error $O(h^2)$. For $N = 64$, $h \approx 0.016$, giving truncation error $O(10^{-4})$. For the smooth boundary conditions used in this project, the actual pointwise error versus the analytical solution is well below 10^{-6} (and empirically measured below 10^{-8} for the sinusoidal benchmark in `tests/test_laplace_solver.py`; see §2.3). This threshold should be verified in code for each new BC family — it reflects the smoothness of the specific test solution, not a general property of the stencil at this resolution.

Three distinct numerical quantities should not be confused:

1. **Error vs analytical solution** (§2.3): $\|T_{\text{FD}} - T_{\text{exact}}\|_\infty$. This measures how well the FD discretization approximates the continuum PDE for a specific BC with a known closed-form solution. Empirically $< 10^{-8}$ for the sinusoidal benchmark at $N = 64$.

2. **Discrete Laplacian residual** (§11.2): $R_{\text{PDE}} = \frac{1}{(N-2)^2} \sum r_{i,j}^2$. This measures how well a field satisfies the discretized equation $L\mathbf{u} = \mathbf{b}$. For the FD solver, this is at or near machine precision — empirically measured at $\sim 10^{-10}$ in the benchmark (Table 2), reflecting accumulated floating-point error in the solver and residual computation. For neural surrogates, this is a meaningful physics compliance metric.
3. **Test tolerances**: Automated tests use explicit tolerance thresholds (e.g., $< 10^{-8}$ for analytical match, $\delta = 10^{-6}$ for maximum principle). These are chosen conservatively above the expected numerical precision.

[Convention.] Throughout this repository, "ground truth" means the FD numerical oracle at fixed resolution $N = 64$ — not the exact continuum solution, except where an analytical benchmark is explicitly available (§2.3). All neural surrogate evaluations are against this numerical oracle.

2.3 Analytical Validation

For the boundary condition $T(x, 0) = 0$, $T(x, 1) = \sin(\pi x)$, $T(0, y) = T(1, y) = 0$, the exact solution is:

```
T_{\text{exact}}(x,y) = \sin(\pi x) \frac{\sinh(\pi y)}{\sinh(\pi)} \tag{2.7}
```

[Derivation.] We seek a separable solution $T(x, y) = X(x)Y(y)$. Substituting into (1.1):

```
X''Y + XY'' = 0 \implies \frac{X''}{X} = -\frac{Y''}{Y} = -\lambda \tag{2.8}
```

The x-equation $X'' + \lambda X = 0$ with $X(0) = X(1) = 0$ has eigenvalues $\lambda_n = n^2\pi^2$ and eigenfunctions $X_n = \sin(n\pi x)$. The y-equation $Y'' - n^2\pi^2 Y = 0$ with $Y(0) = 0$ gives $Y_n = \sinh(n\pi y)$. The top boundary condition $T(x, 1) = \sin(\pi x)$ selects $n = 1$ with coefficient $1/\sinh(\pi)$, yielding (2.7). □

The pointwise agreement $\|T_{\text{FD}} - T_{\text{exact}}\|_\infty < 10^{-8}$ at $N = 64$ supports the solver implementation for this benchmark case. This is a necessary but not sufficient validation — it confirms correctness on one analytical solution, not on all possible BCs. Additional tests (zero BCs, symmetric BCs, maximum principle; see §14.1) provide broader coverage. *Validated in `tests/test_laplace_solver.py`.*

2.4 Boundary Condition Generation

The boundary profile g on each of the four edges is generated to produce a diverse training distribution while maintaining corner consistency.

Corner consistency. [Assumption: corners are shared between adjacent edges.] Each corner of Ω is the endpoint of two edges. For the temperature field to be continuous, these edges must agree at corners. We enforce this by:

1. Sampling 4 corner values $c_1, c_2, c_3, c_4 \sim \text{Uniform}(-1, 1)$.
2. Generating each edge profile to interpolate between its two endpoint corners.

Edge profile construction. For an edge parameterized by $x \in [0, 1]$ with corner values c_{start} and c_{end}:

```
g(x) = \underbrace{c_{\text{start}} + (c_{\text{end}} - c_{\text{start}}) x}_{\text{linear base
```

where $p(x)$ is drawn from one of five families (§2.4.1 below). The envelope $4x(1 - x)$ forces the perturbation to vanish at $x = 0$ and $x = 1$:

```
4 \cdot 0 \cdot (1-0) \cdot p(0) = 0, \quad 4 \cdot 1 \cdot (1-1) \cdot p(1) = 0 \tag{2.10}
```

guaranteeing $g(0) = c_{\text{start}}$ and $g(1) = c_{\text{end}}$ regardless of p. [Exact.]

The factor 4 normalizes the envelope to have maximum value 1 at $x = 1/2$, so that perturbation amplitudes are interpretable as the maximum deviation from the linear baseline at the edge midpoint.

2.4.1 Perturbation Families Five families of perturbation functions $p(x)$ define the BC diversity:

Family 1 — Sinusoidal:

```
p(x) = A \sin(n\pi x), \quad A \sim \text{Uniform}(0.5, 2.0), \quad n \sim \text{Uniform}\{1,2
```

Family 2 — Random Fourier:

```
p(x) = \sum_{k=1}^{K} a_k \sin(k\pi x), \quad a_k \sim \mathcal{N}(0, 1/k^2), \quad K = 5 \tag
```

The $1/k^2$ variance decay produces smooth, band-limited profiles. The Karhunen-Loève-like structure ensures that low-frequency modes dominate, yielding physically plausible boundary conditions.

Family 3 — Gaussian bump:

```
p(x) = A \exp\!\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad A \sim \text{Uniform}(0.5, 3
```

with σ fixed at 0.1. The bump is localized, testing whether surrogates capture local boundary features.

Family 4 — Piecewise constant (held out): Smooth step transitions between 2–3 levels in $[-2, 2]$. This family is **excluded from training** entirely and used only in the OOD test set (`data/test_ood.npz`). It tests whether models generalize beyond the smooth BC families they were trained on.

Family 5 — Linear (no perturbation): $p(x) = 0$. The boundary profile is the linear baseline only. This produces the simplest non-trivial solutions and acts as an easy-case anchor in the training distribution.

Boundary generation is implemented in `src/diffphys/pde/boundary.py`. The held-out split logic is in `pde/generate.py`, controlled by the `allowed_types` parameter.

2.5 The 8-Channel Conditioning Tensor

All neural surrogates receive boundary information through a standardized $(8, 64, 64)$ conditioning tensor. This tensor has two groups of channels:

Value channels (0–3): Each of the four edges (top, bottom, left, right) is encoded as a $(64,)$ profile broadcast into a $(64, 64)$ spatial map: - Channels 0, 1 (top, bottom): profile varies along x-axis, broadcast (tiled) along y-axis. - Channels 2, 3 (left, right): profile varies along y-axis, broadcast (tiled) along x-axis.

Mask channels (4–7): Binary masks indicating which boundary positions are observed vs. interpolated, following the same broadcast convention as the value channels: - Phase 1 (exact BCs): all mask entries are 1.0. - Phase 2 (noisy/sparse BCs): 1.0 at observed positions, 0.0 at interpolated positions.

Design note. The broadcast encoding extends 1D edge information into 2D spatial maps. This is a simple design choice, not a physically exact sensor model — the model must learn to discount the broadcast structure in the interior. A more sophisticated encoding (e.g., attention over sparse observation sets) could improve performance but is not necessary for the benchmark question. The key property is that all models see the same conditioning interface, eliminating architectural confounds.

Mathematically, for the top edge with profile $g_{\text{top}} \in \mathbb{R}^{64}$ and mask $m_{\text{top}} \in \{0,1\}^{64}$:

```
C_0[i,j] = g_{\text{top}}[j], \quad C_4[i,j] = m_{\text{top}}[j] \quad \forall\, i \in \{0,\ldd
```

and for the left edge with profile $g_{\text{left}} \in \mathbb{R}^{64}$ and mask $m_{\text{left}} \in \{0,1\}^{64}$:

```
C_2[i,j] = g_{\text{left}}[i], \quad C_6[i,j] = m_{\text{left}}[i] \quad \forall\, j \in \{0,\l
```

Implemented in `src/diffphys/data/conditioning.py:encode_bcs()`. *Tested in* `tests/test_conditioning.py`

---

3. Neural Surrogate Architectures

3.1 U-Net Regressor (Model 1)

The U-Net is an encoder-decoder convolutional network with skip connections (Ronneberger, Fischer & Brox, 2015). We denote the encoder feature maps at resolution level ℓ as $\mathbf{h}_\ell^{\text{enc}}$ and decoder maps as $\mathbf{h}_\ell^{\text{dec}}$.

Architecture. The encoder applies a sequence of residual blocks and spatial downsampling:

```
\mathbf{h}_0^{\text{enc}} = \text{Conv}_{3 \times 3}(\mathbf{C}; 8 \to 64) \tag{3.1}
```

```
\mathbf{h}_\ell^{\text{enc}} = \text{Down}(\text{ResBlock}(\mathbf{h}_{\ell-1}^{\text{enc}}; c_
```

with channel multipliers $(c_1, c_2, c_3, c_4) = (64, 128, 256, 256)$ and spatial resolutions $(64, 32, 16, 8)$. Self-attention is applied at the 16×16 level ($\ell = 2$).

Residual block. Each ResBlock applies two convolutions with GroupNorm (Wu & He, 2018) and SiLU activation:

```
\text{ResBlock}(\mathbf{x}; c) = \mathbf{x} + \text{Conv}(\text{SiLU}(\text{GN}(\text{Conv}(\te
```

with a 1×1 projection on the skip path if the channel count changes.

Bottleneck. At the coarsest resolution (8×8, 256 channels):

```
\mathbf{h}_{\text{bot}} = \text{ResBlock}(\text{Attn}(\text{ResBlock}(\mathbf{h}_3^{\text{enc}}
```

Decoder. The decoder upsamples and concatenates skip connections:

```
\mathbf{h}_\ell^{\text{dec}} = \text{ResBlock}([\text{Up}(\mathbf{h}_{\ell+1}^{\text{dec}}); \m
```

where $[\cdot\,;\cdot]$ denotes channel-wise concatenation and Up is nearest-neighbor upsampling followed by 3×3 convolution.

Output. A final 3×3 convolution projects to the predicted field:

```
\hat{T} = \text{Conv}_{3 \times 3}(\mathbf{h}_0^{\text{dec}}; 64 \to 1) \tag{3.6}
```

Loss. Mean squared error against the ground-truth field:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N^2} \sum_{i,j} (\hat{T}_{i,j} - T_{i,j}^{\text{true}})^2$$

Implemented in `src/diffphys/model/unet.py` *(backbone) and* `model/regressor.py` *(MSE wrapper). Trained by* `model/train_regressor.py`*. ~5M parameters.*

3.2 Fourier Neural Operator (Model 2)

The FNO learns an operator mapping between function spaces rather than point-to-point regression. Its key inductive bias is the **spectral convolution layer**, which parameterizes the integral kernel in Fourier space.

Spectral convolution. [Following Li et al. (2021).] For input function $v : \Omega \to \mathbb{R}^{d_v}$, the spectral convolution layer computes:

$$(\mathcal{K}v)(x) = \mathcal{F}^{-1}\!\left(R_\phi \cdot \mathcal{F}(v)\right)(x) \tag{3.8}$$

where $\mathcal{F}$ denotes the 2D discrete Fourier transform, $R_\phi \in \mathbb{C}^{k_{\max,1} \times k_{\max,2} \times d_v \times d_v}$ is a learnable complex-valued weight tensor (with $k_{\max}$ modes retained in each spatial frequency dimension), and the product truncates to the lowest $k_{\max}$ Fourier modes per axis.

Why truncation works. For smooth PDE solutions (which Laplace solutions are — they are C^∞ in the interior), the Fourier coefficients decay rapidly. Retaining only $k_{\max} = 16$ modes out of 64 captures the dominant spectral content while regularizing the model against high-frequency artifacts. This is a spectral inductive bias aligned with the smoothness of elliptic PDE solutions.

Full FNO layer. Each layer combines the spectral convolution with a pointwise linear transform:

$$v_{\ell+1}(x) = \sigma\!\left((\mathcal{K}_\ell v_\ell)(x) + W_\ell v_\ell(x)\right) \tag{3.9}$$

where $W_\ell \in \mathbb{R}^{d_v \times d_v}$ is a pointwise (i.e., 1×1 convolution) weight matrix and σ is the GeLU activation. The local path $W_\ell v_\ell$ captures high-frequency residuals not represented by the truncated spectral path.

Architecture summary: 1. Lifting layer: $\text{Conv}_{1\times1}(8 \to 32)$ — maps 8-channel conditioning to width-32 latent 2. 4 spectral convolution layers with $k_{\max} = 16$, width 32 3. Projection layer: $\text{Conv}_{1\times1}(32 \to 1)$ — maps to predicted field

Implemented in `src/diffphys/model/fno.py` *using the* `neuraloperator` *library (Kossaifi et al., 2024) or a minimal custom implementation. ~2M parameters.*

OOD hypothesis. The FNO's spectral parameterization encodes an inductive bias toward smooth, low-frequency solutions. This may confer an advantage on the held-out piecewise BC family, since Laplace solutions are always smooth in the interior regardless of boundary roughness — the FNO's spectral truncation naturally enforces this smoothness. Whether this advantage materializes is an empirical question tested in Table 6.

Empirical outcome. The FNO significantly underperformed expectations, achieving rel. L2 ≈ 0.40 — roughly $40\times$ worse than the U-Net regressor (rel. L2 ≈ 0.01). This is likely due to insufficient model size or training for this specific problem rather than a fundamental limitation of the FNO architecture. The FNO was not tuned further since the project focus is the generative model comparison; it is included as-is as an operator-learning data point.

3.3 Deep Ensemble (Model 3)

Epistemic uncertainty from seed diversity. A deep ensemble consists of $M = 5$ copies of the U-Net regressor (§3.1), each trained independently with a different random seed s_m:

```
\hat{T}_m = f_{\theta_m}(\mathbf{C}), \quad m = 1, \ldots, M \tag{3.10}
```

where θ_m denotes the parameters obtained by training from initialization $\theta_m^{(0)} \sim p(\theta; s_m)$.

Prediction statistics. At inference, the ensemble provides a point estimate and uncertainty:

```
\bar{T}(x,y) = \frac{1}{M} \sum_{m=1}^{M} \hat{T}_m(x,y) \tag{3.11}
```

```
\sigma_T(x,y) = \left(\frac{1}{M-1} \sum_{m=1}^{M} (\hat{T}_m(x,y) - \bar{T}(x,y))^2\right)^{1/
```

What the ensemble captures. [Assumption: ensemble variance approximates epistemic uncertainty.] The variance (3.12) reflects disagreement among models that found different solutions due to different initialization and SGD trajectories. Fort, Hu & Lakshminarayanan (2019) showed that this diversity arises because different random initializations explore distinct basins in function space, not merely different weight-space solutions. This captures *epistemic* uncertainty — uncertainty due to finite training data and model capacity — but not *aleatoric* uncertainty arising from the observation noise itself. Ovadia et al. (2019) demonstrated empirically that deep ensembles performed among the strongest UQ methods under dataset shift across their benchmark suite, outperforming MC-Dropout, variational inference, and other approximate Bayesian methods on most tasks.

What the ensemble does not capture. The ensemble does not sample from a posterior distribution over solution fields. Each member produces a deterministic point estimate; the variance arises from initialization diversity, not from a learned generative process. If the 5 members converge to nearly identical predictions, the ensemble uncertainty collapses to near-zero even if the true posterior is broad. This is the fundamental limitation that conditional generative models (flow matching, DDPM) aim to address.

The calibration baseline. If a conditional generative model (FM or improved DDPM) provides better-calibrated uncertainty than the ensemble at matched sample count (5 vs 5), the interpretation is: "the generative model captures richer posterior structure than seed diversity alone." If the ensemble matches the generative models, the interpretation is: "seed diversity suffices for this problem, and the generative machinery adds cost without benefit." The ensemble + conformal wrapper (§9) provides a separate comparison: post-hoc coverage calibration without generative modeling.

CRPS vs coverage caveat. Note that CRPS rewards both calibration *and* sharpness. An ensemble can achieve low CRPS by producing accurate-on-average point predictions with narrow uncertainty bands — even if those bands frequently miss the truth (low coverage). A generative model with slightly higher CRPS but much higher coverage may be the better uncertainty estimator. When interpreting CRPS comparisons, always check coverage and interval width alongside.

Implemented in `src/diffphys/model/ensemble.py`. *Training orchestrated by* `experiments/train_ensemble.py`.

---

4. Score-Based Diffusion and Langevin Dynamics

This section derives the theoretical foundation of DDPM and its connection to score-based generative modeling. The goal is to explain *what* the denoising network learns (the score function), *how* sampling works (discretized Langevin dynamics), and *what the model does not learn* (the PDE itself).

4.1 The Forward Noising Process

Definition. [Exact; Ho et al. (2020).] Given a clean data sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, the forward process defines a Markov chain of increasingly noisy versions:

```
q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\,\mathbf{x}_
```

for $t = 1, \dots, T$ with a noise schedule $\beta_1, \dots, \beta_T \in (0, 1)$.

Closed-form marginal. [Exact.] By iterating (4.1), the marginal at any step t is:

```
q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0
```

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

[Derivation of (4.2) from (4.1).] At $t = 1$: $\mathbf{x}_1 = \sqrt{\alpha_1}\,\mathbf{x}_0 + \sqrt{1 - \alpha_1}\,\epsilon_1$, with $\epsilon_1 \sim \mathcal{N}(0, \mathbf{I})$. At $t = 2$:

```
\mathbf{x}_2 = \sqrt{\alpha_2}\,\mathbf{x}_1 + \sqrt{1-\alpha_2}\,\boldsymbol{\epsilon}_2 = \s
```

The sum of two independent Gaussians with variances $\alpha_2(1 - \alpha_1)$ and $(1 - \alpha_2)$ gives total variance $\alpha_2 - \alpha_2\alpha_1 + 1 - \alpha_2 = 1 - \alpha_1\alpha_2 = 1 - \bar{\alpha}_2$. By induction, (4.2) holds for all t. □

The reparameterization. Equation (4.2) gives a direct sampling formula:

```
\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\e
```

This is what the training loop uses: sample t uniformly, sample ϵ, compute $\mathbf{x}_t$ via (4.4), and train the network to predict ϵ.

Schedule. We use a linear schedule: β_t interpolates linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$ over $T = 200$ steps. At $t = T$, $\bar{\alpha}_T \approx 0.02$, so $\mathbf{x}_T \approx \mathcal{N}(0, \mathbf{I})$ — the data is nearly destroyed.

4.2 The Reverse Process and Score Function

The reverse diffusion SDE. [Following Song et al. (2021).] The continuous-time limit of the forward process (4.1) is the SDE:

```
d\mathbf{x} = -\frac{1}{2}\beta(t)\,\mathbf{x}\,dt + \sqrt{\beta(t)}\,d\mathbf{w} \tag{4.5}
```

where $\mathbf{w}$ is a standard Wiener process. Anderson (1982) showed that the time-reversal of this SDE is:

```
d\mathbf{x} = \left[-\frac{1}{2}\beta(t)\,\mathbf{x} - \beta(t)\,\nabla_{\mathbf{x}} \log p_t(
```

where $\bar{\mathbf{w}}$ is a reverse-time Wiener process and $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ is the **score function** — the gradient of the log-density of the noisy data distribution at time t.

Connection to Langevin dynamics. The deterministic part of (4.6) includes a term proportional to the score. This is exactly the drift term in **annealed Langevin dynamics**: at each noise level

t, the reverse process follows the score toward high-density regions of p_t, with stochastic noise for exploration. The full reverse trajectory is a noise-annealed MCMC sampler that starts from pure noise ($t = T$) and gradually denoises to a data sample ($t = 0$).

Physical interpretation. The score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ points in the direction of steepest ascent of the log-probability at noise level t. At high t (lots of noise), the score provides a coarse, global signal toward the data manifold. At low t (little noise), the score provides fine-grained, local corrections. This multi-scale denoising is why diffusion models can generate diverse, high-quality samples.

4.3 The ϵ-Prediction Objective

DDPM training objective. [Exact; Ho et al. (2020), Eq. 14.] Rather than estimating the score directly, DDPM trains a network $\epsilon_\phi(\mathbf{x}_t, t)$ to predict the noise ϵ added at step t:

```
\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}}\!\left[\|\bolds
```

where $t \sim \text{Uniform}\{1, \ldots, T\}$, $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, and $\mathbf{x}_t$ is computed via (4.4).

Equivalence to score matching. From (4.2), the conditional score is [Exact]:

```
\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\bar{\a
```

Therefore, predicting ϵ is equivalent to estimating the score up to a known scaling factor [Approximation: the network ϵ_ϕ approximates the *marginal* score $\nabla \log p_t(\mathbf{x}_t)$, not the conditional score $\nabla \log q(\mathbf{x}_t|\mathbf{x}_0)$ which requires knowing $\mathbf{x}_0$]:

```
\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t) \approx -\frac{\boldsymbol{\epsilon}_\phi(\mathbf{
```

The DDPM loss (4.7) is a reweighted version of the denoising score matching objective of Vincent (2011). The DDPM ELBO decomposes into per-timestep KL terms, each of which equals a weighted denoising score matching loss (Ho et al., 2020, Eq. 12); predicting ϵ is equivalent to estimating $-\sqrt{1-\bar{\alpha}_t}\,\nabla_{\mathbf{x}} \log q_t(\mathbf{x})$ via Tweedie's formula. The L_{simple} objective (4.7) drops the ELBO-derived timestep-dependent weights $\beta_t^2/(2\sigma_t^2\alpha_t(1-\bar{\alpha}_t))$ in favor of uniform weighting over t. This is an intentional design choice: Nichol & Dhariwal (2021) confirmed that uniform weighting produces better sample quality than the ELBO-derived weights. The term "reweighted" refers specifically to this replacement, not to an approximation error.

4.4 Conditional Generation

Conditioning mechanism. In our setting, the DDPM generates solution fields T conditioned on boundary observations $\mathbf{C}$. The network receives the 8-channel conditioning tensor concatenated with the noisy field:

```
\boldsymbol{\epsilon}_\phi(\mathbf{x}_t, \mathbf{C}, t): \mathbb{R}^{9 \times 64 \times 64} \t
```

The 9 input channels are: 1 channel for $\mathbf{x}_t$ (the noisy field) and 8 channels for $\mathbf{C}$ (the conditioning tensor from §2.5). The timestep t is injected via sinusoidal positional encoding (Vaswani et al., 2017) into the ResBlocks.

What the model learns. The DDPM learns the conditional score $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t|\mathbf{C})$ — the gradient of the log-density of solution fields at noise level t, given boundary observations $\mathbf{C}$. For exact boundary conditions, the true conditional distribution over solution fields collapses to a delta measure at the unique PDE solution; accordingly, a well-trained conditional DDPM should approach deterministic behavior, with all samples converging to the same field. In practice, residual sampling

variability will persist due to finite model capacity and finite training data — the degree of this variability is itself an indicator of model fit quality. For noisy/sparse observations, the posterior is a genuine distribution over compatible solution fields, and DDPM samples from this posterior.

Critical caveat. The DDPM learns the data distribution $p(\mathbf{x}_0|\mathbf{C})$ induced by the training set, which is generated by the FD solver with a specific prior over boundary profiles (§2.4). It does not learn Laplace's equation in any algebraic sense. If the solver had systematic errors, the DDPM would faithfully reproduce those errors. Physics compliance is an empirical outcome measured by the evaluation metrics (§11), not a property guaranteed by the training objective.

4.5 Sampling (Reverse Process)

DDPM sampling. [Exact; Ho et al. (2020), Algorithm 2.] Starting from $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$, the reverse process iterates:

```
\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar
```

where $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ for $t > 1$ and $\mathbf{z} = 0$ for $t = 1$, and $\sigma_t = \sqrt{\beta_t}$.

Derivation of (4.11). [Exact.] The reverse posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is Gaussian with mean:

```
\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1 - \bar{\alpha}_t}\,\mathbf{x}_0 +
```

Substituting $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\,\epsilon)/\sqrt{\bar{\alpha}_t}$ from (4.4) into (4.12):

```
\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1 - \bar{\alpha}_t} \cdot \frac{\mat
```

Collecting the coefficient of $\mathbf{x}_t$:

```
\frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{(1 - \bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} + \frac{\
```

where we used $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$ and $\beta_t + \alpha_t - \alpha_t \bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t$. The coefficient of ϵ simplifies to $-\beta_t/(\sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_t})$. Replacing ϵ with the network prediction ϵ_ϕ yields (4.11).

Denoised estimate. At any step t, the network's prediction implicitly estimates the clean data:

```
\hat{\mathbf{x}}_0^{(t)} = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\epsilon
```

This denoised estimate is used in the physics regularization (§6). At large t, the estimate is noisy and unreliable. At small t (say $t < T/4$), the estimate is accurate enough to evaluate the PDE residual.

EMA. An exponential moving average of the model weights (Polyak & Juditsky, 1992; Tarvainen & Valpola, 2017) is maintained during training with decay $\gamma = 0.999$:

```
\bar{\theta}_{k+1} = \gamma \bar{\theta}_k + (1-\gamma) \theta_k \tag{4.14}
```

The EMA weights $\bar{\theta}$ are used at inference. This stabilizes sample quality without affecting the training dynamics.

Forward process, reverse sampling, and EMA are implemented in `src/diffphys/model/diffusion.py` *and* `model/sample.py`*. Training loop in* `model/train_ddpm.py`*.*

---

5. Posterior Inference Under Uncertain Observations

5.1 The Observation Model

In Phase 2, the model does not observe the exact boundary profile g. Instead, it receives M noisy point observations per edge:

```
\tilde{g}_i = g(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_{\text{obs}}^2)
```

at positions $x_1, \dots, x_M$ sampled on the edge. More compactly, defining the **observation operator** $\mathcal{H} : T \mapsto (g(x_1), \dots, g(x_M))$ as the boundary trace sampled at the M observation points:

```
\tilde{\mathbf{g}} = \mathcal{H}(T) + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon}
```

The operator $\mathcal{H}$ composes the boundary trace (extracting g from T) with subsampling at the observation positions. The remaining boundary values are reconstructed by linear interpolation between observed points, and the 8-channel mask (§2.5) encodes which positions are directly observed.

What is uncertain. Given sparse noisy observations, the full boundary profile g is unknown. Different boundary completions consistent with (5.1) lead to different PDE solutions. The **posterior** is the distribution over temperature fields T consistent with the observations:

```
p(T | \tilde{\mathbf{g}}) \propto p(\tilde{\mathbf{g}} | T) \cdot p(T) \tag{5.2}
```

where $p(T)$ is the prior over solution fields induced by the BC generation process (§2.4), and $p(\tilde{\mathbf{g}}|T) = \mathcal{N}(\tilde{\mathbf{g}}; \mathcal{H}(T), \sigma_{\text{obs}}^2 \mathbf{I}_M)$ is the Gaussian likelihood from (5.1a).

5.2 Scope of Posterior Claims

[**Assumption: synthetic prior.**] All posterior evaluations in this project are with respect to the *synthetic* prior over boundary profiles defined by the dataset generator (§2.4) and the observation noise model (5.1). The uncertainty reflects what is unknown given the observations under this specific prior — not a general-purpose physics uncertainty estimate.

This distinction matters. If the boundary prior were different (e.g., drawn from real sensor data, or from a different parametric family), the posterior would be different even with the same observations. The DDPM learns to sample from the data-generating posterior, not from a universal posterior over Laplace solutions.

5.3 DDPM as Approximate Posterior Sampler

When trained on the joint distribution of (observations, solutions), the conditional DDPM learns:

```
\mathbf{x}_0^{(k)} \sim p_\phi(\mathbf{x}_0 | \mathbf{C}) \approx p(T | \tilde{\mathbf{g}}), \
```

Drawing K independent samples (each from a different noise initialization $\mathbf{x}_T^{(k)} \sim \mathcal{N}(0, \mathbf{I})$) produces an empirical posterior. The sample diversity reflects the model's learned uncertainty: samples should vary in regions where the observations are sparse or noisy, and agree in regions well-constrained by the data.

Comparison with ensemble. The ensemble's 5 predictions also produce a distribution (§3.3), but its variance reflects initialization diversity, not learned posterior structure. In principle, DDPM should produce richer uncertainty because: 1. Each sample is a full reverse-process trajectory, exploring different modes of the posterior. 2. The diversity is learned from data, not from random initialization artifacts.

Whether this theoretical advantage translates to measurably better calibration at $K = 5$ samples is the central empirical question.

5.4 Training for Variable Observation Quality

[**Design choice.**] Rather than training separate models for each observation regime, a single Phase 2 model handles variable observation quality by randomizing the observation parameters during training:

```
M \sim \text{Uniform}\{8, 12, 16, 24, 32, 48, 64\}, \quad \sigma_{\text{obs}} \sim \text{Unifo
```

Each training sample gets a fresh draw of (M, σ_{obs}). The mask channels in the conditioning tensor (§2.5) communicate the observation quality to the model. At evaluation, fixed regimes are used (Table 5 in the implementation plan).

Why from scratch. All Phase 2 models are trained from scratch — no Phase 1 (exact-BC) weights are reused. Warm-starting would give different models different advantages depending on how well exact-BC features transfer to noisy conditioning, muddying the comparison. The from-scratch rule applies equally to the regressor, FNO, ensemble members, DDPM, improved DDPM, and flow matching.

Observation model implemented in `src/diffphys/data/noise_model.py`*. Variable-quality training controlled by* `data/dataset.py` *with* `observation_model='random'`*.*

---

6. Physics-Informed Regularization

6.1 Motivation

The standard DDPM objective (4.7) is purely data-driven — it matches the score of the empirical data distribution without any knowledge of the PDE. Adding a physics-informed term asks: does explicit enforcement of $\nabla^2 T = 0$ improve sample quality, and at what cost to sample diversity?

Prior work. Bastek, Sun & Kochmann (ICLR 2025) introduced Physics-Informed Diffusion Models (PIDM), which add PDE residual loss computed on the denoised mean estimate $\hat{\mathbf{x}}_0$ during DDPM training, reducing physics residuals by up to two orders of magnitude. Our approach follows the same principle — computing the PDE residual on $\hat{\mathbf{x}}_0^{(t)}$ from (4.13) — but applies it in the specific context of elliptic PDE surrogates with per-sample timestep weighting.

Jensen's Gap caveat. Zhang & Zou (2025) identified a subtlety in this approach: imposing PDE constraints on $\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]$ does not strictly constrain individual samples drawn from $p(\mathbf{x}_0|\mathbf{x}_t)$, because $\nabla^2\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t] = 0$ does not imply $\nabla^2\mathbf{x}_0 = 0$ for each sample (Jensen's inequality). For our benchmark, this means physics regularization may improve the *average* Laplacian residual without guaranteeing that *every* sample satisfies the PDE. We report both mean and per-sample residual statistics to assess this.

6.2 The PDE-Residual Loss

At each training step, we compute the denoised estimate (4.13) and evaluate the discrete Laplacian residual on the interior grid:

```
r_{i,j} = \hat{T}_{i+1,j} + \hat{T}_{i-1,j} + \hat{T}_{i,j+1} + \hat{T}_{i,j-1} - 4\hat{T}_{i,
```

where $\hat{T} = \hat{\mathbf{x}}_0^{(t)}$ from (4.13). The residual field r has shape $(N-2) \times (N-2) = (62, 62)$ for $N = 64$.

The per-sample physics loss:

```
\mathcal{L}_{\text{phys}} = \frac{\sum_{b=1}^{B} w_b \cdot \frac{1}{62^2}\sum_{i,j} r_{i,j}^{(b
```

where B is the batch size, $r^{(b)}$ is the residual for the b-th sample, and the per-sample weight is:

```
w_b = \mathbb{1}[t_b < T/4] \tag{6.3}
```

6.3 Why Per-Sample Weighting

[Critical implementation note.] The timestep t is sampled independently for each element in the batch: $t_b \sim \text{Uniform}\{1, \dots, T\}$ for $b = 1, \dots, B$.

A naive batch-level check `if max(t_batch) < T/4` would almost never activate. For $T = 200$ and $B = 64$, the probability that all 64 samples have $t < 50$ is $(50/200)^{64} = (1/4)^{64} \approx 10^{-39}$. The physics loss would be effectively absent.

Per-sample weighting (6.3) ensures that approximately 25% of samples per batch contribute to $\mathcal{L}_{\text{phys}}$. Only these low-noise samples produce meaningful denoised estimates — at high t, $\hat{\mathbf{x}}_0^{(t)}$ is dominated by noise and its Laplacian residual is uninformative.

6.4 Combined Objective

```
\mathcal{L} = \mathcal{L}_{\text{DDPM}} + \lambda(e) \cdot \mathcal{L}_{\text{phys}} \tag{6.4}
```

with a warmup schedule:

```
\lambda(e) = \begin{cases} 0 & e < e_{\text{warm}} \\ \lambda_{\max} \cdot \frac{e - e_{\text{
```

with $e_{\text{warm}} = 20$, $e_{\text{ramp}} = 20$, $\lambda_{\max} = 0.1$. The warmup period lets the DDPM learn basic denoising before physics regularization is applied.

What this tests. The physics-regularization experiment asks: does explicit PDE enforcement improve the Laplacian residual of generated samples without degrading CRPS or coverage? If yes, physics regularization adds value. If no (e.g., the data-driven score already captures enough PDE structure), then the base generative model is sufficient. Either finding is informative.

Physics regularization is implemented in `model/physics_ddpm.py` with the discrete Laplacian computed by `evaluation/physics.py:discrete_laplacian_batch()`. Code and config exist but this experiment has not yet been trained — no experimental results are available.

---

7. Conditional Flow Matching and Optimal Transport

This section derives the flow matching framework as an alternative to DDPM. Where DDPM learns to denoise via a stochastic reverse SDE (§4), flow matching learns a deterministic velocity field that transports noise to data along straight paths. The connection to optimal transport provides a variational principle analogous to action minimization in classical mechanics.

7.1 The Flow Matching Objective

Continuous normalizing flows. A continuous normalizing flow (CNF) defines a time-dependent velocity field $v_t : \mathbb{R}^d \to \mathbb{R}^d$ that generates a probability path p_t from a source distribution $p_0 = \mathcal{N}(0, \mathbf{I})$ to the data distribution $p_1 \approx q(\mathbf{x})$ via the ODE:

```
\frac{d\mathbf{x}}{dt} = v_t(\mathbf{x}) \tag{7.1}
```

The flow matching (FM) objective (Lipman et al., 2023, Eq. 5) regresses a parametric velocity field v_θ against the true generating velocity u_t:

```
\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim U(0,1),\, \mathbf{x} \sim p_t} \|v_\theta
```

Computing $p_t(\mathbf{x})$ directly is intractable. The key insight is **conditional flow matching** (CFM): condition on a single data point $\mathbf{x}_1$ and define a tractable conditional probability path $p_t(\mathbf{x}|\mathbf{x}_1)$ with conditional velocity $u_t(\mathbf{x}|\mathbf{x}_1)$. Lipman et al. (2023, Theorem 2) prove that the CFM loss

```
\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,\, q(\mathbf{x}_1),\, p_t(\mathbf{x}|\mathbf{x}
```

has identical gradients to (7.2): $\nabla_\theta \mathcal{L}_{\text{FM}} = \nabla_\theta \mathcal{L}_{\text{CFM}}$.

7.2 The Optimal Transport Conditional Path

Gaussian conditional path. Following Lipman et al. (2023, Eqs. 10, 20), the OT-conditional probability path is:

```
p_t(\mathbf{x} | \mathbf{x}_1) = \mathcal{N}\!\left(\mathbf{x};\; t\,\mathbf{x}_1,\; (1 - (1-\
```

As $\sigma_{\min} \to 0$, this produces the **linear interpolant**:

```
\mathbf{x}_t = (1-t)\,\mathbf{x}_0 + t\,\mathbf{x}_1, \quad \mathbf{x}_0 \sim \mathcal{N}(0, \
```

The conditional velocity field (Lipman et al., 2023, Eq. 21) simplifies to:

```
u_t(\mathbf{x} | \mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0 \tag{7.6}
```

This velocity is **constant along each conditional path** — independent of t. Contrast with DDPM, where the noise target ϵ enters the loss with timestep-dependent weighting through the schedule $\bar{\alpha}_t$. The constant velocity makes flow matching targets lower-variance and easier to learn.

The CFM training loop. For each training step: sample $\mathbf{x}_1$ from data, $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$, $t \sim U(0, 1)$; compute $\mathbf{x}_t$ via (7.5); predict $v_\theta(\mathbf{x}_t, \mathbf{C}, t)$ (with conditioning $\mathbf{C}$); compute MSE loss against $u_t = \mathbf{x}_1 - \mathbf{x}_0$.

Implemented in `src/diffphys/model/flow_matching.py` and `model/train_flow_matching.py`. The U-Net backbone from §3.1 is reused with the same architecture; only the training target and sampling procedure change.

Conditioning interface. All conditional models (FM, improved DDPM) use the 8-channel conditioning tensor $\mathbf{C}$ from §2.5 (4 value channels + 4 mask channels), giving `in_channels=9` (1 noisy field + 8 condition). The revised Phase 2 implementation plan's code sketches use `in_channels=6`, which reflects an earlier simplified prototype; the implementation should be updated to match the 8-channel reference specification defined here.

7.3 OT-CFM: Mini-Batch Optimal Transport Coupling

Standard CFM pairs noise $\mathbf{x}_0^{(i)}$ with data $\mathbf{x}_1^{(i)}$ by batch index — the coupling is arbitrary. **OT-CFM** (Tong et al., TMLR 2024) replaces this random coupling with a mini-batch optimal transport plan $\pi(\mathbf{x}_0, \mathbf{x}_1)$, minimizing the total transport cost within each batch:

```
\pi^* = \arg\min_{\pi \in \Pi(\hat{p}_0, \hat{p}_1)} \sum_{i,j} \pi_{ij} \|\mathbf{x}_0^{(i)} -
```

where $\hat{p}_0, \hat{p}_1$ are the empirical distributions of the noise and data batches, and Π is the set of doubly stochastic coupling matrices. For mini-batches of size B, this is solved exactly via the Hungarian algorithm (`scipy.optimize.linear_sum_assignment`) in $O(B^3)$ — negligible overhead for $B = 64$ on 4096-dimensional data.

Implementation note. Tong et al. (TMLR 2024) use exact OT solvers (Hungarian algorithm) by default. Our implementation matches their default: we compute the pairwise squared-L^2 cost matrix and solve for the exact optimal permutation via `linear_sum_assignment`. This produces exact mini-batch OT couplings, not the Sinkhorn approximation used in their Schrödinger Bridge variant (SB-CFM).

Why straighter flows matter. Tong et al. (TMLR 2024, Proposition 3.4) show that when the true OT plan is available, OT-CFM approximates dynamic optimal transport. Empirically (their Table 2), OT-CFM reduces the Normalized Path Energy (NPE) — a measure of path curvature — approaching the Benamou-Brenier minimum. Straighter flows mean fewer ODE solver steps are needed at inference: 50 Euler steps suffice for 64×64 data, versus DDPM's 200 reverse SDE steps.

OT coupling is implemented in `src/diffphys/model/flow_matching.py:OTCouplingMatcher`. Tested in `tests/test_flow_matching.py`.

7.4 Connection to Action Minimization and the Benamou-Brenier Formula

[This section connects the project to the author's mathematical physics background.]

The Benamou-Brenier (2000) dynamic formulation of optimal transport expresses the Wasserstein-2 distance as a variational problem:

```
W_2^2(\mu_0, \mu_1) = \inf_{\rho, v} \int_0^1 \!\!\int_{\mathbb{R}^d} \|v(\mathbf{x}, t)\|^2 \
```

subject to the continuity equation $\partial_t \rho + \nabla \cdot (\rho\, v) = 0$ with boundary conditions $\rho(0) = \mu_0$, $\rho(1) = \mu_1$.

The physics parallel. Equation (7.8) has the structure of an **action functional**: the integrand $\|v\|^2 \rho$ is a kinetic energy density, the continuity equation is a constraint, and the infimum selects the minimum-action path between distributions. This is the measure-theoretic analogue of Hamilton's principle $\delta S = \delta \int L\, dt = 0$, where the Lagrangian $L = \frac{1}{2}\|v\|^2$ is purely kinetic (no potential).

In instanton calculus (the author's thesis topic), one minimizes the Euclidean action $S_E[\phi] = \int \frac{1}{2}|\partial_\mu \phi|^2 + V(\phi)\, d^d x$ to find tunneling paths between vacua. The Benamou-Brenier problem is the $V = 0$ analogue: the "instanton" is the geodesic in Wasserstein space connecting two distributions, and OT-CFM approximates this geodesic with learned velocity fields.

McCann's displacement interpolation. The optimal paths in (7.8) are McCann's displacement interpolations: $\rho_t = ((1-t)\,\mathrm{id} + t\, T_*)_{\#}\mu_0$, where T_* is the Monge map. The OT-CFM interpolant (7.5) discretizes this at the sample level. Lipman et al. (2023, §4.1, Example II) connect their OT paths to McCann interpolation; Tong et al. make the Benamou-Brenier connection explicit through their NPE metric.

7.5 Flow Matching vs DDPM: Structural Comparison

Property	DDPM (§4)	Flow Matching (§7)
Generative process	Reverse SDE (stochastic)	ODE (deterministic)
Training target	Noise ϵ	Velocity $\mathbf{x}_1 - \mathbf{x}_0$
Interpolant	$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$ (nonlinear in t)	$\mathbf{x}_t = (1-t)\,\mathbf{x}_0 + t\,\mathbf{x}_1$ (linear in t)
Target depends on t?	Implicitly (through $\bar{\alpha}_t$ weighting)	No (constant velocity)
Noise schedule needed	Yes (β_t, $\bar{\alpha}_t$ bookkeeping)	No
Sampling steps	200 (SDE)	50 (Euler ODE)
Convergence rate	$O(d)$ (SDE-based)	$O(\sqrt{d})$ (ODE-based, Chen et al., NeurIPS 2023)

The $O(\sqrt{d})$ vs $O(d)$ convergence result for ODE over SDE methods (Chen, Chewi, Lee, Li, Lu & Salim, NeurIPS 2023) provides theoretical motivation for preferring deterministic ODE-based generation. However, this result assumes smooth data distributions with a corrector step and does not directly guarantee that a conditional OT-CFM PDE surrogate will achieve the same dimension-dependent improvement. It is best understood as a directional argument for the ODE framework, not a runtime guarantee for this specific benchmark.

Empirical outcome. On this benchmark, FM's theoretical advantages did not translate into better UQ performance. Improved DDPM (§8) achieved lower functional CRPS on all 5 derived quantities (sparse-noisy regime, matched 5v5) and tighter pixelwise conformal intervals across all observation regimes, despite requiring 4× more sampling steps. On this problem and at this scale, the training improvements (Min-SNR weighting, cosine schedule, v-prediction) appear to have been more impactful than the choice of generative framework. Whether this generalizes to larger-scale or higher-dimensional PDE problems is an open question.

---

8. Improved DDPM Training

This section documents three modifications to the standard DDPM training pipeline (§4) that are motivated by prior work reporting substantially faster and more stable diffusion training. These are applied to produce an improved DDPM as a comparison point against flow matching.

8.1 Cosine Noise Schedule

Motivation. The linear schedule $\beta_t \in [10^{-4}, 0.02]$ (Ho et al., 2020) destroys signal too quickly for low-resolution data. At 64×64, over half the information is lost by $t = T/3$.

Cosine schedule (Nichol & Dhariwal, ICML 2021, Eq. 16):

```
\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2\!\left(\frac{t/T + s}{1 + s} \cdot \fra
```

with $s = 0.008$ chosen so that $\sqrt{\beta_0}$ is slightly smaller than the pixel bin size 1/127.5. The betas are $\beta_t = 1 - \bar{\alpha}_t/\bar{\alpha}_{t-1}$, clipped to at most 0.999.

8.2 Zero-Terminal SNR

The problem. Common schedules fail to enforce $\bar{\alpha}_T = 0$, leaving residual signal at the final timestep. Lin et al. (WACV 2024, Eq. 10) showed that for Stable Diffusion, $\sqrt{\bar{\alpha}_T} = 0.068$, meaning $\mathbf{x}_T = 0.068\,\mathbf{x}_0 + 0.998\,\epsilon$ — the model sees leaked low-frequency content during training but starts from pure $\mathcal{N}(0, \mathbf{I})$ at inference.

The fix. Rescale the schedule to enforce $\bar{\alpha}_T = 0$ exactly (Lin et al., 2024, Algorithm 1), so $\mathrm{SNR}(T) = \bar{\alpha}_T/(1 - \bar{\alpha}_T) = 0$. Combined with v-prediction (§8.3), this eliminates the training-inference mismatch.

The cosine schedule with zero-terminal SNR is implemented in `src/diffphys/model/diffusion.py:cosine_beta_`

8.3 v-Prediction Parameterization

Definition (Salimans & Ho, ICLR 2022, §4). Instead of predicting noise ϵ, the network predicts:

```
v_t = \sqrt{\bar{\alpha}_t} \cdot \boldsymbol{\epsilon} - \sqrt{1 - \bar{\alpha}_t} \cdot \mat
```

Why v-prediction is more stable. With ϵ-prediction, recovering $\mathbf{x}_0$ requires $\hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\epsilon})/\sqrt{\bar{\alpha}_t}$, which divides by $\sqrt{\bar{\alpha}_t} \to 0$ at high noise — amplifying prediction errors. v-prediction avoids this: $\hat{\mathbf{x}}_0 = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{v}_t$, which remains well-conditioned across all timesteps. The v-prediction loss yields an effective weighting of $1 + \mathrm{SNR}(t)$, which assigns nonzero weight even at $\mathrm{SNR} = 0$ — unlike ϵ-prediction whose weight vanishes there.

8.4 Min-SNR-$\gamma_{\mathbf{SNR}}$ Loss Weighting

The conflict. Different timesteps produce conflicting gradients: low-noise steps push toward fine details while high-noise steps push toward coarse structure. Uniform weighting (as in L_{simple}, Eq. 4.7) lets high-noise steps dominate.

Min-SNR-$\gamma_{\mathbf{SNR}}$ (Hang et al., ICCV 2023, §3.4) clips the signal-to-noise ratio to reduce the contribution of high-noise timesteps (we write γ_{SNR} to distinguish from the EMA decay γ in §4.5):

```
\text{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \quad w(t) = \frac{\min(\text{SNR}(t
```

With $\gamma_{\text{SNR}} = 5$ (the paper's recommended default): timesteps where $\mathrm{SNR} < 5$ receive full weight; timesteps where $\mathrm{SNR} > 5$ are downweighted. The weighted loss is:

```
\mathcal{L}_{\text{MinSNR}} = \mathbb{E}_{t}\!\left[w(t) \cdot \|\hat{v}_t - v_t\|^2\right] \ta
```

Hang et al. report a **3.4× speedup** in reaching FID 10 compared to previous weighting strategies.

Combined effect. Cosine schedule + zero-terminal SNR + v-prediction + Min-SNR-γ_{SNR} address complementary failure modes of standard DDPM training. Prior work reports up to 3.4× faster convergence from Min-SNR weighting alone (Hang et al., ICCV 2023).

Empirical outcome. The standard DDPM at 60 epochs achieved only 16.8% raw coverage at the 90% target — the loss was still decreasing and the model had not learned the full noise-level spectrum. With the three training improvements, 80 epochs of improved DDPM achieved 85–99% raw coverage across all observation regimes (in-distribution), and the lowest functional CRPS on all 5 derived quantities in the sparse-noisy regime at matched 5-sample count. On this benchmark, the improved DDPM also outperformed flow matching on most metrics despite using 4× more sampling steps. These results are from single training runs (see §16 for replication scope).

All three improvements are implemented in `src/diffphys/model/diffusion.py` *and* `model/train_ddpm.py` *via config flags. Tested in* `tests/test_diffusion.py`*.*

---

9. Conformal Prediction for Calibrated Uncertainty

This section derives the conformal prediction framework. Standard split conformal prediction provides **distribution-free, finite-sample coverage guarantees**; the pooled-pixel variant used in this project's benchmark is a practical adaptation for field-valued outputs (see exchangeability caveat in §9.1). Unlike DDPM and flow matching, which learn approximate posteriors, conformal prediction wraps any base predictor with a calibration adjustment — at zero additional training cost. The pixelwise variant (§9.1) is the primary method benchmarked in this project; the spatial variant (§9.2) is derived as an additional option for applications requiring simultaneous field-level coverage.

9.1 Pixelwise Split Conformal Prediction

Setup (Vovk et al., 2005; Lei et al., 2018). Given exchangeable calibration data $\{(\mathbf{X}_i, Y_i)\}_{i=1}^n$ and a pre-trained model $\hat{f}$ with uncertainty estimate $\hat{\sigma}$, define the **normalized nonconformity score** (Angelopoulos & Bates, 2021, §2.3.2):

```
R_i = \frac{|Y_i - \hat{f}(\mathbf{X}_i)|}{\hat{\sigma}(\mathbf{X}_i)} \tag{9.1}
```

The conformal quantile is the $\lceil (n+1)(1-\alpha) \rceil / n$-th empirical quantile of $\{R_1, \dots, R_n\}$:

```
\hat{q} = \text{Quantile}\!\left(\{R_i\}_{i=1}^n,\; \frac{\lceil(n+1)(1-\alpha)\rceil}{n}\righ
```

The prediction interval for a new test point $\mathbf{X}_{n+1}$ is:

```
C(\mathbf{X}_{n+1}) = \left[\hat{f}(\mathbf{X}_{n+1}) - \hat{q}\,\hat{\sigma}(\mathbf{X}_{n+1}
```

Coverage guarantee (Vovk et al., 2005, Ch. 2; Angelopoulos & Bates, 2021, Theorem 1). For exchangeable data:

```
1 - \alpha \;\leq\; \mathbb{P}(Y_{n+1} \in C(\mathbf{X}_{n+1})) \;\leq\; 1 - \alpha + \frac{1}{
```

This is a **finite-sample, distribution-free** guarantee. It holds regardless of the quality of $\hat{f}$ or the distribution of the data — only exchangeability is required. [Exact.]

Pixelwise application to PDE fields. For PDE surrogate outputs, we apply a practical pooled-pixel conformal variant: all $n \times N^2$ pixel-level nonconformity scores are pooled across calibration samples and spatial locations, and the conformal quantile $\hat{q}$ is computed from this pooled set. This is intended to approximate **marginal pixelwise calibration** — producing tighter intervals than the spatial variant (§9.2) — and is the variant used in the benchmark results (Tables 3, 5).

Exchangeability caveat. The textbook split-conformal guarantee (9.4) assumes exchangeable calibration scores. Pooling across spatial locations introduces dependence: pixels within the same field are spatially correlated, so the $n \times N^2$ pooled scores are not fully exchangeable. In practice, the large pool size ($n \times N^2 \gg 1$) and the averaging over many independent calibration fields make the procedure well-behaved empirically, but the formal finite-sample guarantee (9.4) does not strictly apply to this pooled construction. The reported near-nominal 90% coverage across regimes is an empirical observation, not a rigorous theoretical guarantee.

9.2 Spatial Conformal Prediction (Additional Variant)

For applications requiring coverage at **all spatial locations simultaneously**, motivated by multivariate-output conformal prediction (Feldman et al., JMLR 2023), we define the **spatial maximum** nonconformity score:

```
R_i^{\text{spatial}} = \max_{(m,n)} \frac{|T_i^{\text{true}}(m,n) - \bar{T}_i(m,n)|}{\sigma_i(m
```

where $\bar{T}_i$ and σ_i are the ensemble mean and standard deviation for calibration sample i. Applying split conformal prediction (9.2) to the scalar scores $\{R_i^{\text{spatial}}\}$ yields a prediction band:

```
C_{\text{spatial}}(\mathbf{X}) = \left\{T : |T(m,n) - \bar{T}(m,n)| \leq \hat{q} \cdot \sigma(m
```

with the guarantee $\mathbb{P}(T_{n+1}^{\text{true}} \in C_{\text{spatial}}(\mathbf{X}_{n+1})) \geq 1-\alpha$, where inclusion means the bound holds at all pixels simultaneously. This guarantee is valid but **conservative by construction**: calibrating to the worst-case pixel produces wider bands than necessary for most spatial locations. Both variants are implemented; the spatial variant was not included in the primary benchmark results.

9.3 Conformal Prediction vs Generative Posterior Sampling

Property	Generative (DDPM / FM)	Conformal (Ensemble + CP)
Coverage guarantee	Approximate (learned)	Exact for standard split CP; pooled-pixel variant here is empirical
Training cost	High (GPU hours)	Zero (post-hoc wrapper)
Interval shape	Implicit (sample diversity)	Explicit $[\mu \pm \hat{q}\sigma]$
Requires calibration set	No	Yes (split from val)
Adapts to observation quality	Via conditioning	Via $\hat{\sigma}$ variation

Complementarity with CRPS. The Ferro (2014) fair CRPS (§11.4) evaluates calibration **post-hoc** as a proper scoring rule. Standard split conformal prediction guarantees coverage **by construction** for exchangeable data; the pooled-pixel variant used here is motivated by that framework but evaluated empirically. They answer different questions: CRPS asks "how well-calibrated is this model's uncertainty?"; conformal prediction says "here is a prediction band calibrated to achieve target coverage."

Important distinction. The conformal guarantee applies to the *prediction band* $C(\mathbf{X})$, not to the *ensemble posterior* itself. If the ensemble's uncertainty estimates $\hat{\sigma}$ are poorly correlated with actual errors, the conformal band will be wide (to compensate); for standard split conformal the coverage guarantee holds, while for the pooled-pixel variant used here coverage is assessed empirically. Conformal prediction fixes the coverage of the output interval; it does not retroactively calibrate the base model's probabilistic predictions.

Empirical outcome (pixelwise conformal, in-distribution, all regimes). Pixelwise conformal prediction lifted all methods to near-nominal 90% coverage across all observation regimes. But the interval widths varied substantially: under sparse-noisy conditions, DDPM pixelwise conformal intervals averaged 0.088, versus 0.131 (FM) and 0.135 (ensemble). This confirms that the raw uncertainty quality — not the conformal wrapper — determines the practical utility of the prediction

bands. Results are from single training runs.

Implemented in `src/diffphys/evaluation/conformal.py`. *Tested in* `tests/test_conformal.py`.

---

10. Diffusion Posterior Sampling

This section derives the DPS framework for PDE inverse problems. Unlike the conditional models in §4 and §7, which require boundary observations as input during both training and inference, DPS uses an **unconditional** generative prior and incorporates observations via gradient guidance at inference time. This enables zero-shot adaptation to new observation patterns.

10.1 The Inverse Problem Formulation

Given sparse noisy boundary observations $\tilde{\mathbf{g}} = \mathcal{H}(T) + \varepsilon$ from §5.1, the posterior is:

```
p(T | \tilde{\mathbf{g}}) \propto \underbrace{p(T)}_{\text{prior}} \cdot \underbrace{p(\tilde{
```

DPS (Chung et al., ICLR 2023) decomposes the posterior score via Bayes' rule:

```
\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t | \tilde{\mathbf{g}}) = \underbrace{\nabla_{\mathbf
```

The unconditional score is estimated by a diffusion/flow model trained on clean Laplace solutions without boundary condition input. The likelihood score requires marginalizing over $p_t(\mathbf{x}_0|\mathbf{x}_t)$, which is intractable.

10.2 The DPS Approximation

Core approximation (Chung et al., ICLR 2023, Algorithm 1). Replace the intractable marginalization with a point estimate at the Tweedie denoised mean:

```
p_t(\tilde{\mathbf{g}} | \mathbf{x}_t) \approx p(\tilde{\mathbf{g}} | \hat{\mathbf{x}}_0), \qua
```

For Gaussian observations $\tilde{\mathbf{g}}|T \sim \mathcal{N}(\mathcal{H}(T), \sigma_{\text{obs}}^2\mathbf{I})$, the likelihood gradient becomes:

```
\nabla_{\mathbf{x}_t} \log p_t(\tilde{\mathbf{g}} | \mathbf{x}_t) \approx -\frac{1}{\sigma_{\te
```

10.3 Dual Guidance: Measurement + Physics

Following DiffusionPDE (Huang et al., NeurIPS 2024, Eq. 8), we extend the DPS update with a **physics guidance** term:

```
\mathbf{x}_{t-1} = \text{reverse\_step}(\mathbf{x}_t) - \zeta_{\text{obs}} \nabla_{\mathbf{x}_
```

The measurement term $\mathcal{L}_{\text{obs}}$ pushes samples toward consistency with the noisy observations. The physics term $\mathcal{L}_{\text{pde}}$ pushes samples toward the Laplace solution manifold. The guidance strengths $\zeta_{\text{obs}}, \zeta_{\text{pde}}$ are annealed during sampling (stronger early, weaker late).

For flow matching, the denoised estimate at ODE step i (out of N steps) is:

```
\hat{\mathbf{x}}_0 \approx \mathbf{x}_t + (1 - t)\,v_\theta(\mathbf{x}_t, t) \tag{10.6}
```

which replaces the DDPM Tweedie estimate (10.3). The guidance gradients (10.4)–(10.5) apply identically.

10.4 Why DPS Matters for Deployment

The key advantage of DPS over conditional models: **a single unconditional model handles any observation pattern without retraining.** If the sensor layout changes — different number of observation points, different noise level, observations on only some edges — the conditional models (§4, §7) must be retrained with the new observation model. DPS adapts at inference time by changing only the observation operator $\mathcal{H}$ in (10.4).

Theoretical status. The original DPS paper (Chung et al., ICLR 2023) provides no formal convergence guarantees — the Tweedie approximation (10.3) is justified empirically. For related unconditional-prior inverse-problem theory, Xu & Chi (NeurIPS 2024) introduced the DPnP framework — a different algorithm that alternates between a proximal consistency sampler and a denoising diffusion sampler — and proved asymptotic consistency under exact scores and diminishing step sizes (their Theorem 1), with non-asymptotic error bounds showing convergence to a distorted posterior π_η with total variation error $O(\eta + \epsilon_{\text{score}} + \epsilon_{\text{solver}})$ (their Theorem 2). These results do not directly apply to the DPS update rule (10.5), but they establish that provably convergent posterior sampling with unconditional diffusion priors is achievable in principle.

DPS is a stretch goal. If implemented, the unconditional flow matching model uses `model/unet.py` *with* `in_channels=1` *(no conditioning). The guidance sampler is in* `model/dps_sampler.py`*. Tested in* `tests/test_dps.py`*.*

---

11. Evaluation Theory: Proper Scoring and Calibration

11.1 Deterministic Metrics

Mean Squared Error (MSE):

```
\text{MSE} = \frac{1}{N^2} \sum_{i,j} (\hat{T}_{i,j} - T_{i,j}^{\text{true}})^2 \tag{11.1}
```

Relative L^2 error:

```
\text{Rel-}L^2 = \frac{\|\hat{T} - T^{\text{true}}\|_2}{\|T^{\text{true}}\|_2} = \frac{\left(\s
```

Maximum pointwise error:

```
\text{MaxErr} = \max_{i,j} |\hat{T}_{i,j} - T_{i,j}^{\text{true}}| \tag{11.3}
```

11.2 Physics Compliance Metrics

Laplacian residual. The mean squared PDE residual over the interior:

```
R_{\text{PDE}} = \frac{1}{(N-2)^2} \sum_{i,j} r_{i,j}^2 \tag{11.4}
```

where $r_{i,j}$ is from (6.1). For the FD numerical oracle, R_{PDE} is empirically $\sim 10^{-10}$ (Table 2), reflecting accumulated floating-point error in the LU solve and residual computation — the field is the exact solution of the discrete linear system $L\mathbf{u} = \mathbf{b}$ to machine precision, but the residual computation amplifies rounding errors. This is the discrete residual, not the truncation error vs the continuum solution (see the three-quantity distinction in §2.2). For neural surrogates, R_{PDE} measures how well the prediction satisfies $\nabla^2 T = 0$ — for unconstrained surrogate predictions and for physics-regularized generative variants (§6).

Boundary condition error:

```
E_{\text{BC}} = \frac{1}{4N} \sum_{\text{edges}} \sum_{k} |\hat{T}_k^{\text{edge}} - g_k^{\tex
```

Maximum principle violations. Fraction of interior points violating (2.1):

```
V_{\text{MP}} = \frac{1}{(N-2)^2} \sum_{i,j} \mathbb{1}\!\left[\hat{T}_{i,j} < \min_{\partial\O
```

with tolerance $\delta = 10^{-6}$ to absorb numerical noise.

Dirichlet energy functional:

```
\mathcal{E}[T] = \frac{1}{2} \int_\Omega |\nabla T|^2 \, dA \approx \frac{h^2}{2} \sum_{i,j} \l
```

The Dirichlet energy is minimized by harmonic functions (solutions of Laplace's equation). Comparing $\mathcal{E}[\hat{T}]$ to $\mathcal{E}[T^{\text{true}}]$ tests whether the surrogate's prediction is near the energy minimum.

All metrics implemented in `src/diffphys/evaluation/accuracy.py` *and* `evaluation/physics.py`*. Tested in* `tests/test_physics_metrics.py`*.*

11.3 Derived Physical Quantities (Functionals)

Beyond pixel-level comparison, we evaluate uncertainty on derived quantities that a practitioner might care about. Each functional maps a solution field to a scalar:

Center temperature:

```
\mathcal{Q}_1[T] = T(0.5, 0.5) \tag{11.8}
```

(bilinear interpolation if the center falls between grid points).

Subregion mean temperature:

```
\mathcal{Q}_2[T] = \frac{1}{|S|} \int_S T \, dA, \quad S = [0.25, 0.75]^2 \tag{11.9}
```

Maximum interior temperature:

```
\mathcal{Q}_3[T] = \max_{(x,y) \in \Omega^\circ} T(x,y) \tag{11.10}
```

(a nonlinear functional — harder to predict uncertainty for).

Dirichlet energy $\mathcal{Q}_4 = \mathcal{E}[T]$ from (11.7).

Top-edge normal derivative (heat flux proxy):

```
\mathcal{Q}_5[T] = \int_0^1 \left.\frac{\partial T}{\partial y}\right|_{y=1} dx \approx h \sum_
```

Convention note. $\mathcal{Q}_5$ is the integrated outward normal derivative $\partial T/\partial n$ at the top edge (where $\hat{n} = +\hat{y}$), not the physical conductive heat flux $q_n = -k\,\partial T/\partial n$. Since we work in dimensionless units with $k = 1$, the physical flux is $-\mathcal{Q}_5$. We report the normal derivative directly; the sign is a convention that does not affect CRPS or coverage evaluation.

These functionals test whether the surrogate's uncertainty is well-calibrated for quantities that depend on different aspects of the solution: smooth interior averages ($\mathcal{Q}_2$), pointwise values ($\mathcal{Q}_1$), extrema ($\mathcal{Q}_3$), global integrals ($\mathcal{Q}_4$), and boundary derivatives ($\mathcal{Q}_5$).

Hypothesis. Conditional generative models (FM or improved DDPM) may outperform the ensemble on boundary-sensitive quantities ($\mathcal{Q}_5$, $\mathcal{Q}_4$) because they generate diverse boundary completions, while the ensemble may suffice for smooth interior quantities ($\mathcal{Q}_2$). If standard split conformal

were applied to scalar functionals, it would provide finite-sample coverage under exchangeability, though that is not the primary benchmark reported here. These are testable predictions.

Empirical outcome (sparse-noisy regime, matched 5v5, in-distribution). Improved DDPM achieved the best (lowest) functional CRPS on all 5 derived quantities at matched 5-sample count. The advantage was largest on boundary-sensitive quantities: Dirichlet energy CRPS = 0.127 (DDPM) vs 0.190 (ensemble) vs 0.276 (FM), and top-edge flux CRPS = 0.044 (DDPM) vs 0.112 (ensemble) vs 0.069 (FM). The hypothesis was partially confirmed: generative models did outperform on boundary-sensitive quantities, but DDPM also won on smooth interior quantities ($\mathcal{Q}_2$), indicating a broader advantage in this regime. Results are from single training runs; see §16 for replication scope.

Functionals implemented in `src/diffphys/evaluation/functionals.py`*. Tested in* `tests/test_functionals.p`

11.4 CRPS: The Continuous Ranked Probability Score

Definition. [Exact; Gneiting & Raftery (2007).] The CRPS for a predictive distribution F and observation y is:

```
\text{CRPS}(F, y) = \int_{-\infty}^{\infty} (F(z) - \mathbb{1}[z \geq y])^2 \, dz \tag{11.12}
```

CRPS is a **proper scoring rule**: it is minimized in expectation when the predictive distribution F equals the true data-generating distribution. It simultaneously rewards calibration (the predictive CDF should match the empirical frequency of outcomes) and sharpness (narrower intervals score better, conditional on calibration).

Ensemble estimator. For an ensemble of K samples $X_1, \dots, X_K$, the population CRPS identity $\mathbb{E}|X - y| - \frac{1}{2}\mathbb{E}|X - X'|$ (Gneiting & Raftery, 2007, Eq. 21) is estimated by the U-statistic (Ferro, 2014):

```
\text{CRPS} = \frac{1}{K}\sum_{k=1}^{K} |X_k - y| - \frac{1}{2K(K-1)}\sum_{k=1}^{K}\sum_{l=1}^{
```

This is the "fair" CRPS estimator of Ferro (2014), which uses $K(K-1)$ in the denominator of the second term rather than K^2, correcting a negative bias in the spread term present in the naive V-statistic estimator. The V-statistic *underestimates* the spread (pairwise) term by a factor of $(K-1)/K$ because it divides by K^2 instead of $K(K-1)$; in the loss orientation, this causes the naive CRPS to be biased high (forecasts appear less sharp than they are). At $K = 5$, the correction changes the denominator from 25 to 20. We use the loss orientation (lower is better), which is the negated form of the positive-score convention in Gneiting & Raftery (2007).

Why CRPS over log-likelihood. Log-likelihood requires a density estimate, which is hard to construct reliably from 5 samples. CRPS operates directly on samples and is well-defined even for small K. It also has the same units as the quantity being predicted, making it interpretable.

Usage. For each derived quantity $\mathcal{Q}$, compute $\mathcal{Q}[T^{(k)}]$ for each of K samples (ensemble members, FM draws, or DDPM draws) and the truth $\mathcal{Q}[T^{\text{true}}]$, then evaluate (11.13). Lower CRPS is better.

CRPS computation in `src/diffphys/evaluation/scoring.py:compute_crps()`*. Tested in* `tests/test_scoring.py`*.*

11.5 Pixel-Level Coverage

Definition. The $(1-\alpha)$ coverage rate is the fraction of pixels where the truth falls within the sample-based prediction interval:

```
\text{Cov}_{1-\alpha} = \frac{1}{N^2} \sum_{i,j} \mathbb{1}\!\left[Q_{\alpha/2}^{(i,j)} \leq T_
```

where $Q_p^{(i,j)}$ is the p-th quantile of the K sample values at pixel (i, j).

Quantile estimator convention. With only $K = 5$ or 10 samples, quantile estimation is coarse. We use NumPy's default linear interpolation method (`numpy.quantile` with `method='linear'`, corresponding to Hyndman & Fan method 7). This method computes the virtual index as $q \times (K-1)$ and linearly interpolates between the two adjacent order statistics. For $K = 5$: the 0.05 quantile has virtual index $0.05 \times 4 = 0.2$, interpolating between the 1st and 2nd sorted samples (80% weight on the minimum, 20% on the next value); the 0.95 quantile has virtual index $0.95 \times 4 = 3.8$, interpolating between the 4th and 5th sorted samples (20%/80% weights). The 90% prediction interval therefore does not span the full sample range but is slightly narrower than $[\min, \max]$. This discreteness means that pixel-level coverage estimates have inherent sampling noise; reported coverage values should be understood as approximate, and small deviations from the nominal level (e.g., 88% vs 90%) may not be meaningful.

For perfect calibration, $\text{Cov}_{0.9} = 0.9$. Under-coverage (< 0.9) indicates overconfident uncertainty; over-coverage (> 0.9) indicates overly conservative uncertainty.

Empirical outcome (pixelwise coverage, 20 generative samples, in-distribution). Ensemble raw coverage at the 90% target degrades severely as observation quality decreases: 82% (exact) → 55% (dense-noisy) → 31% (sparse-noisy) → 15% (very-sparse). Generative models (using 20 samples for mean/std estimation) maintain 84–99% raw coverage across all regimes. After pixelwise conformal calibration, all methods achieve near-nominal 90% coverage, but the ensemble requires the widest intervals to compensate for its poor raw calibration. Note: the 20-sample generative evaluation gives FM/DDPM a statistical advantage over the 5-member ensemble; matched pixel-level comparison at 5 samples is pending.

Interval width. The mean width of the 90% prediction interval:

```
\overline{W}_{0.9} = \frac{1}{N^2} \sum_{i,j} \left(Q_{0.95}^{(i,j)} - Q_{0.05}^{(i,j)}\right)
```

Sharper intervals (smaller $\overline{W}$) are better, conditional on correct coverage.

Uncertainty-error correlation. The Pearson correlation between pixel-level ensemble/sample standard deviation and absolute error:

```
\rho = \text{corr}\!\left(\sigma_{i,j}, |T_{i,j}^{\text{true}} - \bar{T}_{i,j}|\right) \tag{11
```

A high ρ indicates that the model "knows what it doesn't know" — uncertainty is concentrated where errors are large.

Implemented in `src/diffphys/evaluation/uncertainty.py`. *Tested in* `tests/test_uncertainty.py`.

11.6 Sample Count Fairness

The matched-sample protocol. All core uncertainty comparisons use matched sample counts across the three Phase 2 UQ methods:

- **Generative vs generative:** 20 FM samples vs 20 improved-DDPM samples (matched compute budget, different generative frameworks).
- **Generative vs ensemble:** 5 ensemble members vs 5 FM/DDPM samples, and 5 ensemble members vs 20 FM/DDPM samples (the first tests sample-for-sample value of the generative machinery; the second tests whether additional generative samples improve on the ensemble).
- **Ensemble vs conformal:** raw ensemble coverage vs conformalized ensemble coverage at matched 5 members (tests the value of the conformal wrapper at zero additional training cost).

A conditional generative model (FM or DDPM) is considered meaningfully better than the ensemble **only if** it improves calibration or CRPS at matched-5. If improvement appears only at 20 samples, this is reported as a higher-cost tradeoff, since generative models require substantially more inference FLOPs per sample than an ensemble forward pass.

This protocol is pre-committed to avoid post-hoc rationalization: "FM is better if you use enough samples" is a weaker claim than "FM is better sample-for-sample."

Current status. Functional-level CRPS (§11.3) uses matched 5v5 as the primary comparison. Pixel-level coverage and interval-width metrics currently use 20 samples for generative models vs 5 ensemble members, giving generative models a statistical advantage in mean/std estimation. A matched pixel-level comparison at 5 samples is a planned follow-up.

Matched-sample evaluation logic in `experiments/evaluate_phase2.py`.

11.7 Calibration Diagrams

Reliability diagrams provide a visual complement to the scalar coverage metric. For a set of nominal confidence levels $p \in \{0.1, 0.2, \ldots, 0.9\}$, compute the empirical coverage — the fraction of test pixels where the truth falls within the p-level prediction interval. Plot empirical coverage (y-axis) vs nominal level (x-axis). A perfectly calibrated model lies on the diagonal.

Reliability diagrams are computed separately for the ensemble, ensemble+conformal, FM, and improved DDPM, and for selected derived quantities (center temperature $\mathcal{Q}_1$ and Dirichlet energy $\mathcal{Q}_4$). For nominal levels at which conformal sets are explicitly calibrated, empirical coverage should be on or above the diagonal up to finite-sample variability. Over-confidence appears as a curve below the diagonal; over-conservatism as a curve above.

Implemented in `src/diffphys/evaluation/calibration.py`. *Figures generated by* `experiments/generate_figu`

---

12. Reference Paper Connections

12.1 Ho, Jain & Abbeel (2020) — "Denoising Diffusion Probabilistic Models"

- **We use:** The DDPM framework — forward noising (4.1), ϵ-prediction objective (4.7), reverse sampling (4.11), and linear β schedule.
- **Their Eq. (14):** $L_{\text{simple}} = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2]$
- **Our Eq. (4.7):** Same, with the addition of conditioning $\mathbf{C}$ concatenated to the input.
- **Differences:** We use $T = 200$ (they use $T = 1000$). Our U-Net is smaller (~5M vs ~35.7M parameters for their CIFAR10 model; their 256×256 models use 114M). We add conditional

input channels. These reduce computational cost at the expense of sample quality, which is acceptable for a benchmark study.

12.2 Song, Sohl-Dickstein, Kingma et al. (2021) — "Score-Based Generative Modeling through Stochastic Differential Equations"

- **We use:** The continuous-time perspective (§4.2) connecting DDPM to score matching and Langevin dynamics. Equations (4.5)–(4.6) and the score–ϵ equivalence (4.8)–(4.9).
- **Differences:** We do not implement their SDE solvers or the probability flow ODE. We use the discrete DDPM reverse process (4.11) only. The continuous-time view is used for theoretical understanding, not for sampling.

12.3 Lakshminarayanan, Pritzel & Blundell (2017) — "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles"

- **We use:** The deep ensemble approach for non-generative probabilistic prediction (§3.3).
- **Differences:** Their recipe includes adversarial training (FGSM-based, but marked optional in their Algorithm 1 and shown not to help significantly on their benchmarks) and Gaussian NLL loss with learned variance $\sigma_\theta^2(\mathbf{x})$. We use standard MSE training with different seeds only — a simpler setup that provides a clean baseline. If the ensemble underperforms, the MSE-only training (which captures only epistemic uncertainty, not learned aleatoric variance) is a possible explanation.

12.4 Li, Kovachki, Azizzadenesheli et al. (2021) — "Fourier Neural Operator for Parametric Partial Differential Equations"

- **We use:** The spectral convolution layer (3.8) and FNO architecture (§3.2).
- **Their Eq. (3):** Defines the general kernel integral operator $\int_D \kappa(x, y, a(x), a(y); \phi)\, v_t(y)\, dy$.
- **Their Eq. (4):** Parameterizes the kernel directly in Fourier space: $(\mathcal{K} v_t)(x) = \mathcal{F}^{-1}(R_\phi \cdot (\mathcal{F} v_t))(x)$.
- **Our Eq. (3.8):** Same as their Eq. (4) — the Fourier-space parameterization, not the general kernel.
- **Differences:** We use $k_{\max} = 16$ modes and width 32 (smaller than their settings for Darcy flow, which used 12 modes and width 32). Our problem (2D Laplace) is simpler than their benchmarks (Navier-Stokes, Darcy flow). The FNO is included to test whether spectral inductive bias improves generalization to the held-out BC family.

12.5 Gneiting & Raftery (2007) — "Strictly Proper Scoring Rules, Prediction, and Estimation"

- **We use:** CRPS as the primary probabilistic evaluation metric (§11.4).
- **Their Eq. (21):** The population-level energy form identity $\mathrm{CRPS}(F, x) = \mathbb{E}|X - x| - \frac{1}{2}\mathbb{E}|X - X'|$, where X, X' are independent draws from F. This is an identity for a distribution F, not a finite-sample formula.
- **Our Eq. (11.13):** The finite-sample (U-statistic) estimator obtained by replacing expectations with averages over K samples. For $K = 5$, this is an approximation to the population CRPS, not an exact evaluation.
- **Differences:** Gneiting & Raftery use a positive-score orientation (higher is better; the CRPS is negated relative to our convention). We use the loss orientation standard in ML (lower

is better), which is the negated form: CRPS $= \frac{1}{K}\sum|X_k - y| - \frac{1}{2K(K-1)}\sum|X_k - X_l|$. The mathematical content is identical; only the sign convention differs.

12.6 Evans (2010) — *Partial Differential Equations*, 2nd ed. (AMS Graduate Studies in Mathematics, Vol. 19)

- **We use:** Existence, uniqueness, and strong maximum principle for Laplace's equation (§2.1), from §2.2.3 of the text (Theorems 4 and 5 in the editions we have consulted; readers should verify numbering against their own copy, as theorem labels may vary between printings).
- **Our application:** The maximum principle provides a physics-based evaluation metric (2.1) that is independent of the training objective. It tests whether neural surrogates respect the elliptic structure of the PDE.
- **Differences:** None — we apply standard results from elliptic PDE theory.

12.7 Vincent (2011) — "A Connection Between Score Matching and Denoising Autoencoders"

- **We use:** The equivalence between the denoising autoencoder training criterion and score matching (§4.3). This foundational result justifies interpreting the DDPM ϵ-prediction loss (4.7) as a reweighted denoising score matching objective, connecting the noise prediction network to the score function via (4.8)–(4.9).
- **Their result:** A denoising autoencoder trained with Gaussian noise implicitly performs score matching against a Parzen density estimator of the data.
- **Our application:** We do not implement denoising autoencoders directly, but the theoretical equivalence underpins why ϵ-prediction works as a score estimator. Vincent's result is the bridge between the DDPM loss and the Langevin dynamics interpretation of §4.2.

12.8 Anderson (1982) — "Reverse-Time Diffusion Equation Models"

- **We use:** The reverse-time SDE result (§4.2, Eq. 4.6), which establishes that a forward diffusion process admits a reverse-time SDE whose drift depends on the score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$.
- **Our application:** This classical result provides the mathematical foundation for all score-based generative models. We cite it through the lens of Song et al. (2021), who apply it to the specific VP-SDE used in DDPM.

12.9 Ronneberger, Fischer & Brox (2015) — "U-Net: Convolutional Networks for Biomedical Image Segmentation"

- **We use:** The encoder-decoder architecture with skip connections (§3.1). The U-Net's spatial pyramid structure and skip connections enable multi-scale feature extraction, which is why it became the standard backbone for diffusion models.
- **Differences:** Our U-Net is adapted for conditional field prediction (8 or 9 input channels, 1 output channel) rather than biomedical segmentation. We add attention layers, GroupNorm, and sinusoidal time embeddings following the DDPM convention.

12.10 Wu & He (2018) — "Group Normalization"

- **We use:** GroupNorm in all ResBlocks (§3.1, Eq. 3.3). GroupNorm is preferred over BatchNorm in diffusion models because it does not depend on batch statistics, which vary with the random timestep sampling across the batch.

12.11 Vaswani et al. (2017) — "Attention Is All You Need"

- **We use:** Sinusoidal positional encoding for the diffusion timestep t (§4.4). The encoding maps the scalar timestep to a high-dimensional vector via $\mathrm{PE}(t, 2i) = \sin(t/10000^{2i/d})$, $\mathrm{PE}(t, 2i+1) = \cos(t/10000^{2i/d})$, which is then projected and added to the ResBlock features.
- **Differences:** Vaswani et al. encode token *position* in a sequence; we encode diffusion *timestep*. The functional form is identical.

12.12 Polyak & Juditsky (1992) / Tarvainen & Valpola (2017) — EMA of Model Weights

- **We use:** Exponential moving average of model parameters during training (§4.5, Eq. 4.14). Polyak & Juditsky (1992) established the theoretical foundation for iterate averaging in stochastic optimization. Tarvainen & Valpola (2017) popularized EMA of network weights in deep learning ("Mean Teachers"), and it is now standard practice in diffusion model training.

12.13 Ferro (2014) — "Fair Scores for Ensemble Forecasts"

- **We use:** The bias-corrected ("fair") CRPS estimator for finite ensembles (§11.4, Eq. 11.13). The naive V-statistic (dividing the pairwise term by K^2) *underestimates* the spread term, causing the CRPS to be biased high in the loss orientation. Ferro's U-statistic (dividing by $K(K-1)$) corrects this bias.
- **Our application:** With $K = 5$ samples, the bias correction changes the denominator from 25 to 20 — a 20% difference in the spread term. This matters for the matched-sample comparison.

12.14 Bastek, Sun & Kochmann (ICLR 2025) — "Physics-Informed Diffusion Models"

- **We use:** The concept of adding PDE-residual loss computed on the denoised estimate $\hat{\mathbf{x}}_0$ during DDPM training (§6). Bastek et al. introduced two variants: PIDM-ME (mean estimation, applying physics loss to $\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]$) and PIDM-SE (single-step estimation).
- **Differences:** Our implementation uses per-sample timestep weighting (Eq. 6.3) to restrict the physics loss to low-noise steps, and applies it to the 2D Laplace equation specifically. Bastek et al. demonstrate results on reaction-diffusion, Navier-Stokes, and Kolmogorov flow.
- **Jensen's Gap:** Zhang & Zou (2025) identified that physics constraints on $\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]$ do not strictly constrain individual samples. We acknowledge this caveat in §6.1.

12.15 Ovadia et al. (2019) — "Can You Trust Your Model's Uncertainty?"

- **We use:** Empirical evidence that deep ensembles performed among the strongest UQ methods under dataset shift (§3.3). This strengthens the choice of ensembles as the calibration baseline.

12.16 LeVeque (2007) — *Finite Difference Methods for Ordinary and Partial Differential Equations*

- **We use:** The standard 5-point finite difference stencil for the 2D Laplacian (§2.2, Eq. 2.2). Chapter 3 derives the stencil and its $O(h^2)$ truncation error.

12.17 Nichol & Dhariwal (2021) — "Improved Denoising Diffusion Probabilistic Models"

- **We use:** Confirmation that the uniform-weighted L_{simple} objective produces better sample quality than the ELBO-derived weights (§4.3). This justifies our use of (4.7) rather than the full variational bound.

12.18 Song, Meng & Ermon (2021) — "Denoising Diffusion Implicit Models" (DDIM)

- **We use:** DDIM as a stretch-goal extension (Day 12 buffer). DDIM enables deterministic sampling via a non-Markovian reverse process with 50 steps instead of 200, reducing inference cost by 4×.
- **Differences:** Our primary results use the standard DDPM reverse process (4.11). DDIM would be an optional faster-inference row in Table 7.

12.19 Lipman, Chen, Ben-Hamu, Nickel & Le (ICLR 2023) — "Flow Matching for Generative Modeling"

- **We use:** The conditional flow matching framework (§7). Theorem 2 (CFM has identical gradients to FM); Eqs. (10, 20) for the Gaussian conditional path; Eq. (22) for the OT flow map; Eq. (23) for the reparameterized CFM loss.
- **Our Eq. (7.3):** Same as their Eq. (9), with conditional velocity from their Eq. (21).
- **Differences:** We use $\sigma_{\min} \to 0$ throughout (straight interpolant). They present multiple path families; we use only the OT path.

12.20 Tong, Fatras, Malkin et al. (TMLR 2024) — "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport"

- **We use:** OT-CFM (§7.3), which replaces random noise-data pairing with mini-batch optimal transport coupling. Proposition 3.4 connects OT-CFM to dynamic optimal transport.
- **Differences:** Our implementation matches their default: exact OT via the Hungarian algorithm (`linear_sum_assignment`), not the Sinkhorn approximation from their Schrödinger Bridge variant.

12.21 Benamou & Brenier (2000) — "A Computational Fluid Mechanics Solution to the Monge-Kantorovich Mass Transfer Problem"

- **We use:** The dynamic OT formulation (§7.4, Eq. 7.8) connecting flow matching to action minimization. The Benamou-Brenier variational problem $\inf \int \|v\|^2 \rho \, dx \, dt$ subject to the continuity equation provides the physics bridge between OT-CFM and the instanton/action-minimization language from the author's mathematical physics background.

12.22 Hang et al. (ICCV 2023) — "Efficient Diffusion Training via Min-SNR Weighting Strategy"

- **We use:** Min-SNR-γ_{SNR} loss weighting (§8.4, Eq. 8.3) with $\gamma_{\text{SNR}} = 5$. The weight $w(t) = \min(\text{SNR}(t), \gamma_{\text{SNR}})/\text{SNR}(t)$ resolves conflicting gradients across timesteps, yielding ~3.4× faster convergence.

12.23 Salimans & Ho (ICLR 2022) — "Progressive Distillation for Fast Sampling of Diffusion Models"

- **We use:** The v-prediction parameterization (§8.3, Eq. 8.2). v-prediction avoids the division by $\sqrt{\bar{\alpha}_t} \to 0$ that makes ϵ-prediction numerically unstable at high noise.

12.24 Lin et al. (WACV 2024) — "Common Diffusion Noise Schedules and Sample Steps are Flawed"

- **We use:** Zero-terminal SNR enforcement (§8.2). When $\bar{\alpha}_T \neq 0$, a training-inference mismatch arises because the model sees leaked low-frequency signal at $t = T$ during training but pure noise at inference.

12.25 Vovk, Gammerman & Shafer (2005) / Lei et al. (2018) — Split Conformal Prediction

- **We use:** The finite-sample coverage guarantee (§9.1, Eq. 9.4): $\mathbb{P}(Y_{n+1} \in C(X_{n+1})) \geq 1 - \alpha$. Vovk et al. (2005) originated conformal prediction; Lei et al. (2018) formalized the split conformal framework.

12.26 Angelopoulos & Bates (2021) — "A Gentle Introduction to Conformal Prediction"

- **We use:** The normalized nonconformity score $R_i = |Y_i - \hat{f}(X_i)|/\hat{\sigma}(X_i)$ (§9.1, Eq. 9.1) and the adaptive prediction interval (§9.1, Eq. 9.3). Their Theorem 1 provides the two-sided coverage bound.

12.27 Ma, Pitt, Azizzadenesheli & Anandkumar (TMLR 2024) — "Calibrated UQ for Operator Learning via Conformal Prediction"

- **We use:** Their work as motivation for applying conformal prediction to neural operator outputs (§9). Ma et al. propose a risk-controlling quantile neural operator with a functional calibration guarantee on the coverage rate. Our spatial maximum nonconformity score (§9.2, Eq. 9.5) is a different construction — based on the multivariate conformal framework of Feldman et al. (JMLR 2023) — but addresses the same problem: calibrated uncertainty for function-valued predictions.

12.28 Chung et al. (ICLR 2023) — "Diffusion Posterior Sampling for General Noisy Inverse Problems"

- **We use:** The DPS approximation (§10.2, Eq. 10.3): replace the intractable likelihood marginalization with a point estimate at the Tweedie denoised mean. The gradient guidance update (Eq. 10.4–10.5) decomposes posterior sampling into prior (unconditional model) + likelihood (measurement + physics gradients).
- **Theoretical status:** No formal convergence guarantees in the original DPS paper. Xu & Chi (NeurIPS 2024) proved asymptotic consistency for a related algorithm (DPnP), establishing that provably convergent posterior sampling with unconditional diffusion priors is achievable. Their results are for DPnP specifically, not for the DPS gradient-guidance update.

12.29 Huang et al. (NeurIPS 2024) — "DiffusionPDE: Generative PDE-Solving Under Partial Observation"

- **We use:** Dual guidance framework (§10.3, Eq. 10.5) combining measurement loss $\mathcal{L}_{\text{obs}}$ and PDE residual loss $\mathcal{L}_{\text{pde}}$. They train on joint distributions of PDE coefficients and solutions.

12.30 Chen, Chewi, Lee, Li, Lu & Salim (NeurIPS 2023) — "The Probability Flow ODE is Provably Fast"

- **We use:** The $O(\sqrt{d})$ convergence rate for ODE-based sampling vs $O(d)$ for SDE-based DDPM (§7.5). This provides the strongest theoretical motivation for preferring flow matching over DDPM.

---

13. Theory-to-Deliverable Mapping

Theory Section	Key Equations	Repository Deliverable	Application Claim
§2.1 Maximum principle	(2.1)	`evaluation/physics.py:physics_maximum_principle()`	Physics-aware evaluation beyond pixel MSE
§2.2 FD discretization	(2.2)–(2.6)	`pde/laplace.py:LaplaceSolver`	Ground-truth oracle for surrogate evaluation
§2.3 Analytical solution	(2.7)–(2.8)	`tests/test_laplace_solver.py`	Solver validation against known exact solution
§2.4 BC generation	(2.9)–(2.13)	`pde/boundary.py`, `pde/generate.py`	Controlled dataset with held-out OOD family
§2.5 Conditioning tensor	(2.14)–(2.15)	`data/conditioning.py:encode_bc()`	Standardized input interface for all models
§3.1 U-Net architecture	(3.1)–(3.7)	`model/unet.py`, `model/regressor.py`	Deterministic baseline
§3.2 FNO spectral conv	(3.8)–(3.9)	`model/fno.py`	Operator-learning baseline with spectral bias
§3.3 Deep ensemble	(3.10)–(3.12)	`model/ensemble.py`	Probabilistic non-generative calibration baseline
§4.1–4.3 DDPM theory	(4.1)–(4.9)	`model/diffusion.py`, `model/train_ddpm.py`	Score-based generative surrogate
§4.4 Conditional generation	(4.10)	`model/unet.py` (9-channel input)	Conditioning on boundary observations
§4.5 Reverse sampling	(4.11)–(4.14)	`model/sample.py`, `model/diffusion.py`	Posterior sampling under uncertain BCs

Theory Section	Key Equations	Repository Deliverable	Application Claim
§5.1 Observation model	(5.1)	`data/noise_model.py`	Noisy/sparse boundary observation regime
§5.4 Variable-quality training	(5.4)	`data/dataset.py` (random obs mode)	Single model for variable observation quality
§6.2–6.4 Physics regularization	(6.1)–(6.5)	`model/physics_ddpm.py`	PDE-informed generative model (not yet trained)
§11.1 Deterministic metrics	(11.1)–(11.3)	`evaluation/accuracy.py`	Standard accuracy evaluation
§11.2 Physics compliance	(11.4)–(11.7)	`evaluation/physics.py`	Physics-aware evaluation suite
§11.3 Derived functionals	(11.8)–(11.11)	`evaluation/functionals.py`	UQ on quantities practitioners care about
§11.4 CRPS	(11.12)–(11.13)	`evaluation/scoring.py` `compute_crps()`	Proper scoring rule for posterior quality
§11.5 Coverage & width	(11.14)–(11.16)	`evaluation/uncertainty.py`	Pixel-level posterior calibration
§11.6 Sample fairness	— (protocol)	`experiments/evaluate_phase2.py`	Phase 2 matched-sample comparison
§11.7 Calibration diagrams	— (visual)	`evaluation/calibration.py`	Reliability plots for all UQ methods
§2.4 + §11 OOD evaluation	(2.1), Table 6 protocol	`evaluation/generalization.py`	Held-out BC family stress test
§11.1–11.5 Benchmark orchestration	— (pipeline)	`evaluation/benchmark.py`	Orchestrate all evaluations → JSON results
— Phase 1 experiment	—	`experiments/evaluate_phase1.py`	Phase 1 OOD evaluation of Phase 1 models
— Training orchestration	—	`experiments/train_fno.py`, `experiments/train_ddpm.py`, `experiments/train_flow_matching.py`	GPU training scripts (Modal)
— Integration test	—	`tests/test_pipeline_integration.py`	End-to-end generate → train → evaluate
§7.1–7.6 Flow matching theory	(7.1)–(7.8)	`model/flow_matching.py`, `model/train_flow_matching.py`	OT-CFM generative backbone
§7.3 OT coupling	(7.7)	`model/flow_matching.py` `OTCouplingMatcher`	Mini-batch optimal transport
§8.1–8.4 Improved DDPM	(8.1)–(8.4)	`model/diffusion.py` (cosine schedule, v-pred, Min-SNR)	Improved DDPM comparison point

Theory Section	Key Equations	Repository Deliverable	Application Claim
§9.1–9.2 Conformal prediction	(9.1)–(9.6)	`evaluation/conformal.py`, `experiments/evaluate_conformal.py`	Post-hoc coverage calibration for field uncertainty
§10.1–10.3 DPS (stretch)	(10.1)–(10.6)	`model/dps_sampler.py` (if implemented)	Unconditional prior + guidance

Orphan check: All core mathematical components (PDE theory, model architectures, diffusion theory, evaluation metrics) map to at least one deliverable. Orchestration, configuration, and figure-generation utilities (`experiments/run_all.py`, `experiments/generate_figures.py`, `configs/`) are omitted from this table as they do not implement mathematical content. Training scripts for individual models (`experiments/train_*.py`) and the integration test are included above for completeness but are wrappers around the core `src/diffphys/` modules.

13.1 Evidence Mapping: Empirical Claims to Result Artifacts

Major empirical claims made in the theory document are traceable to the following result artifacts:

Claim (section)	Result artifact	Evaluation script
DDPM achieves lowest functional CRPS on all 5 quantities (§11.3)	`docs/benchmark_results.md` Table 4	`experiments/evaluate_phase2.py --eval-type functional-crps`
Ensemble raw coverage degrades 82%→15% across regimes (§11.5)	`docs/benchmark_results.md` Table 5	`experiments/evaluate_phase2.py --eval-type phase2-all`
Pixelwise conformal lifts all methods to ~90% coverage (§9.3)	`docs/benchmark_results.md` Tables 3, 5	`experiments/evaluate_conformal.py`
DDPM conformal intervals tightest (0.088 vs 0.131/0.135, sparse-noisy) (§9.3)	`docs/benchmark_results.md` Table 5	`experiments/evaluate_conformal.py`
FM underperforms DDPM on functional CRPS despite simpler training (§7.5)	`docs/benchmark_results.md` Table 4	`experiments/evaluate_phase2.py --eval-type functional-crps`
Improved DDPM achieves 85–99% raw coverage across regimes (§8)	`docs/benchmark_results.md` Table 5	`experiments/evaluate_phase2.py --eval-type phase2-all`
Standard DDPM at 60 epochs achieved 16.8% raw coverage (§8, diagnostic)	Phase 1 diagnostic run (not in benchmark tables)	`experiments/diagnose_ddpm.py`
FNO underperforms U-Net by ~40× on rel. L2 (§3.2)	`docs/benchmark_results.md` Table 1	`experiments/evaluate_phase1.py`
OOD: DDPM shows lower reported calibration error than ensemble across regimes	`docs/benchmark_results.md` Table 6	`experiments/evaluate_phase2.py --eval-type ood-regimes`

Claim (section)	Result artifact	Evaluation script
Training convergence: FM vs improved DDPM (§8)	`figures/fig10_convergence.png`	`experiments/generate_figures.py`

All result numbers cited in the theory document are from single training runs (one checkpoint per model). See §16 closing note for replication scope.

---

14. Validation Criteria

14.1 Solver Validation

Test	Expected	Tolerance	Script
Analytical match: $T(x, y) = \sin(\pi x)\sinh(\pi y)/\sinh(\pi)$	$\lVert T_{\text{FD}} - T_{\text{exact}} \rVert_\infty$	$< 10^{-8}$	`tests/test_laplace_solver.py`
Zero BCs → zero interior	All interior values = 0	Machine precision	`tests/test_laplace_solver.py`
Symmetric BCs → symmetric solution	$T(x, y) = T(1 - x, y)$ for symmetric top/bottom	$< 10^{-10}$	`tests/test_laplace_solver.py`
Maximum principle	All interior in $[\min g, \max g]$	$\delta = 10^{-6}$	`tests/test_laplace_solver.py`
Corner consistency	Edge endpoints match corner values	Exact	`tests/test_boundary.py`
Solver reuse	Same factorization object across calls; identical BCs → bitwise-identical results; different BCs → different results	Identity + equality checks	`tests/test_laplace_solver.py`

14.2 Conditioning Validation

Test	Expected	Script
Shape: exact mode	$(8, 64, 64)$	`tests/test_conditioning.py`
Shape: noisy mode	$(8, 64, 64)$	`tests/test_conditioning.py`
Exact masks = all ones	$C_{4:8} = 1.0$	`tests/test_conditioning.py`
Top channel constant along y	$C_0[0, :] = C_0[63, :]$	`tests/test_conditioning.py`
Left channel constant along x	$C_2[:, 0] = C_2[:, 63]$	`tests/test_conditioning.py`

Test	Expected	Script
Zero noise, full obs exact	$\|C_{\text{noisy}} - C_{\text{exact}}\| < 10^{-10}$	`tests/test_conditioning.py`

14.3 Model Validation

Test	Expected	Script
U-Net output shape (8ch input)	$(B, 1, 64, 64)$	`tests/test_unet.py`
U-Net output shape (9ch input)	$(B, 1, 64, 64)$	`tests/test_unet.py`
Gradient flow through U-Net	All params have non-None grad	`tests/test_unet.py`
U-Net param count	4M–8M	`tests/test_unet.py`
FNO output shape	$(B, 1, 64, 64)$	`tests/test_fno.py`
DDPM forward process at $t = T$	Per-pixel: mean ≈ 0 (< 0.1), std ≈ 1 ($\in [0.8, 1.2]$), on a batch of ≥ 100 samples	`tests/test_diffusion.py`
DDPM loss decreases on small data	Deterministic overfit test: fixed seed, 10 samples, 20 epochs $\rightarrow$ final loss $< 0.1\times$ initial loss	`tests/test_smoke_ddpm.py`
Regressor loss decreases on small data	Deterministic overfit test: fixed seed, 10 samples, 20 epochs $\rightarrow$ final loss $< 0.1\times$ initial loss	`tests/test_smoke_regressor.py`

14.4 Metric Validation

Test	Expected	Script
MSE of identical fields	0	`tests/test_physics_metrics.py`
Laplacian residual of FD solution	$< 10^{-8}$ (discrete residual near machine precision; empirically $\sim 10^{-10}$ in benchmark)	`tests/test_physics_metrics.py`
Max principle: FD solution	0 violations	`tests/test_physics_metrics.py`
CRPS: perfect prediction	0	`tests/test_scoring.py`
CRPS: wider ensemble > narrower	Higher CRPS for dispersed samples	`tests/test_scoring.py`
Coverage: truth inside all intervals	100%	`tests/test_uncertainty.py`

14.5 Flow Matching Validation

Test	Expected	Script
Interpolant at $t = 0$ equals noise $\mathbf{x}_0$	$\|\mathbf{x}_t - \mathbf{x}_0\| = 0$	`tests/test_flow_matching.py`
Interpolant at $t = 1$ equals data $\mathbf{x}_1$	$\|\mathbf{x}_t - \mathbf{x}_1\| = 0$	`tests/test_flow_matching.py`
Velocity target is constant: $u_t = \mathbf{x}_1 - \mathbf{x}_0$	Independent of t	`tests/test_flow_matching.py`
OT coupling is a permutation of the batch	Each output row matches one input row (valid because implementation uses exact Hungarian solver)	`tests/test_flow_matching.py`
ODE sampling output shape	$(B, 1, 64, 64)$	`tests/test_flow_matching.py`
FM loss decreases on small data	Deterministic overfit test: fixed seed	`tests/test_flow_matching.py`

14.6 Conformal Prediction Validation

Test	Expected	Script
Calibration sets $\hat{q} > 0$	Positive quantile threshold	`tests/test_conformal.py`
Prediction intervals widen with higher $\hat{\sigma}$	Monotonicity of interval width	`tests/test_conformal.py`
Coverage $\geq 1 - \alpha$ on synthetic exchangeable data	$\geq 89\%$ at $\alpha = 0.1$ (finite-sample slack)	`tests/test_conformal.py`
Uncalibrated predictor raises error	`RuntimeError` before `calibrate()`	`tests/test_conformal.py`
Zero uncertainty handled without NaN	All outputs finite	`tests/test_conformal.py`

14.7 Improved DDPM Validation

Test	Expected	Script
Cosine schedule: $\bar{\alpha}_T = 0$ after zero-terminal SNR rescaling	$\bar{\alpha}_T < 10^{-10}$	`tests/test_diffusion.py`
Cosine schedule: initial cumulative alpha ≈ 1	$f(0)/f(0) = 1$ (by definition); first $\bar{\alpha}_1 > 0.999$	`tests/test_diffusion.py`
v-prediction roundtrip: `recover_x0_from_v(compute_v_target(x_0, eps, abar))` $\approx x_0$	$< 10^{-6}$	`tests/test_diffusion.py`
Min-SNR weights: all finite, positive, and ≤ 1	`assert (w > 0).all() and (w <= 1).all()`	`tests/test_diffusion.py`

Test	Expected	Script
Min-SNR weights: $w(t) \approx 1$ when $\mathrm{SNR}(t) \leq \gamma_{\mathrm{SNR}}$	$\lvert w(t) - 1 \rvert < 10^{-6}$	`tests/test_diffusion.py`
Improved DDPM loss decreases on small data	Deterministic overfit: fixed seed, 10 samples, 20 epochs	`tests/test_smoke_ddpm.py`

---

15. Notation Reference

Symbol	Meaning	Units / Range
$T(x, y)$	Temperature field	dimensionless (normalized)
Ω	Spatial domain $[0, 1]^2$	—
$g(x, y)$	Boundary profile	dimensionless
N	Grid points per dimension	64
h	Grid spacing $1/(N-1)$	≈ 0.016
L	Sparse Laplacian matrix	$(N-2)^2 \times (N-2)^2$
B_L	Tridiagonal block of L	$(N-2) \times (N-2)$
B	Batch size (training context)	typically 64
$\mathbf{C}$	8-channel conditioning tensor	$(8, 64, 64)$
$\mathcal{H}$	Observation operator (boundary trace + subsample)	$T \mapsto \mathbb{R}^M$
$\mathbf{x}_t$	Noisy field at diffusion step t	$(1, 64, 64)$
ϵ	Gaussian noise	$\sim \mathcal{N}(0, \mathbf{I})$
ϵ_ϕ	Noise prediction network	$(1, 64, 64) \to (1, 64, 64)$
α_t	$1 - \beta_t$	$(0, 1)$
$\bar{\alpha}_t$	$\prod_{s=1}^{t} \alpha_s$	$(0, 1)$
β_t	Noise schedule	$[10^{-4}, 0.02]$
T (diffusion)	Number of diffusion steps	200
M	Observation points per edge	$\{8, \dots, 64\}$
σ_{obs}	Observation noise std	$[0, 0.2]$
K	Number of posterior samples	5 (matched functional CRPS) or 20 (pixel-level UQ)
$\mathcal{Q}[T]$	Derived functional (scalar)	varies
CRPS	Continuous ranked probability score	same units as $\mathcal{Q}$
R_ϕ	FNO spectral weights	$\mathbb{C}^{k_{\max,1} \times k_{\max,2} \times d_v \times d_v}$
$k_{\max}$	FNO mode truncation	16

Symbol	Meaning	Units / Range
λ	Physics regularization weight	$[0, 0.1]$
$r_{i,j}$	Discrete Laplacian residual at (i, j)	dimensionless
γ	EMA decay	0.999
γ_{SNR}	Min-SNR clipping threshold	5.0
v_θ	Learned velocity field (flow matching)	$(1, 64, 64) \to (1, 64, 64)$
u_t	Target velocity $\mathbf{x}_1 - \mathbf{x}_0$	$(1, 64, 64)$
π	OT coupling matrix	$\mathbb{R}_{\geq 0}^{B \times B}$, doubly stochastic
W_2	Wasserstein-2 distance	≥ 0
v_t (DDPM)	v-prediction target $\sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1 - \bar{\alpha}_t}\mathbf{x}_0$	$(1, 64, 64)$
$\text{SNR}(t)$	Signal-to-noise ratio $\bar{\alpha}_t/(1 - \bar{\alpha}_t)$	$[0, \infty)$
R_i	Nonconformity score for calibration sample i	≥ 0
$\hat{q}$	Conformal quantile threshold	> 0
α (conformal)	Miscoverage rate	typically 0.1
$\zeta_{\text{obs}}, \zeta_{\text{pde}}$	DPS guidance strengths	> 0

Convention notes.

1. The symbol T is overloaded: it denotes both the temperature field $T(x, y)$ and the total number of diffusion steps. Context always disambiguates — T with spatial arguments is the temperature; T in diffusion-process equations (or as a subscript bound like $t = 1, \ldots, T$) is the step count.

2. Array indexing: $T[i, j] = T(y_i, x_j)$ — axis 0 is vertical (y), axis 1 is horizontal (x). See §2.2 for the full convention.

---

16. Bibliography

1. **Anderson, B. D. O.** (1982). Reverse-Time Diffusion Equation Models. *Stochastic Processes and their Applications*, 12(3):313–326. DOI: 10.1016/0304-4149(82)90051-5.

2. **Bastek, J.-H., Sun, W. J., & Kochmann, D. M.** (2025). Physics-Informed Diffusion Models. *ICLR 2025*. arXiv:2403.14404.

3. **Evans, L. C.** (2010). *Partial Differential Equations*, 2nd ed. AMS Graduate Studies in Mathematics, Vol. 19. ISBN: 978-0-8218-4974-3.

4. **Ferro, C. A. T.** (2014). Fair Scores for Ensemble Forecasts. *Q. J. R. Meteorol. Soc.*, 140(683):1917–1923. DOI: 10.1002/qj.2270.

5. **Fort, S., Hu, H., & Lakshminarayanan, B.** (2019). Deep Ensembles: A Loss Landscape Perspective. arXiv:1912.02757.

6. **Gneiting, T. & Raftery, A. E.** (2007). Strictly Proper Scoring Rules, Prediction, and Estimation. *JASA*, 102(477):359–378. DOI: 10.1198/016214506000001437.

7. **Ho, J., Jain, A., & Abbeel, P.** (2020). Denoising Diffusion Probabilistic Models. *NeurIPS 2020*. arXiv:2006.11239.

8. **Lakshminarayanan, B., Pritzel, A., & Blundell, C.** (2017). Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. *NeurIPS 2017*. arXiv:1612.01474.

9. **LeVeque, R. J.** (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM. DOI: 10.1137/1.9780898717839.

10. **Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A.** (2021). Fourier Neural Operator for Parametric Partial Differential Equations. *ICLR 2021*. arXiv:2010.08895.

11. **Nichol, A. & Dhariwal, P.** (2021). Improved Denoising Diffusion Probabilistic Models. *ICML 2021*. arXiv:2102.09672.

12. **Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J. V., Lakshminarayanan, B., & Snoek, J.** (2019). Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift. *NeurIPS 2019*. arXiv:1906.02530.

13. **Polyak, B. T. & Juditsky, A. B.** (1992). Acceleration of Stochastic Approximation by Averaging. *SIAM J. Control Optim.*, 30(4):838–855. DOI: 10.1137/0330046.

14. **Ronneberger, O., Fischer, P., & Brox, T.** (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI 2015*, LNCS 9351, pp. 234–241. arXiv:1505.04597.

15. **Song, J., Meng, C., & Ermon, S.** (2021). Denoising Diffusion Implicit Models. *ICLR 2021*. arXiv:2010.02502.

16. **Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B.** (2021). Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR 2021* (Outstanding Paper Award). arXiv:2011.13456.

17. **Tarvainen, A. & Valpola, H.** (2017). Mean Teachers Are Better Role Models: Weight-Averaged Consistency Targets Improve Semi-Supervised Learning Results. *NeurIPS 2017*. arXiv:1703.01780.

18. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I.** (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762.

19. **Vincent, P.** (2011). A Connection Between Score Matching and Denoising Autoencoders. *Neural Computation*, 23(7):1661–1674. DOI: 10.1162/NECO_a_00142.

20. **Wu, Y. & He, K.** (2018). Group Normalization. *ECCV 2018*, LNCS 11217, pp. 3–19. arXiv:1803.08494.

21. **Zhang, Y. & Zou, D.** (2025). Physics-Informed Distillation of Diffusion Models for PDE-Constrained Generation. arXiv:2505.22391.

22. **Angelopoulos, A. N. & Bates, S.** (2021). A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511. Also in *Foundations and Trends in Machine Learning*, Now Publishers, 2023.

23. **Benamou, J.-D. & Brenier, Y.** (2000). A Computational Fluid Mechanics Solution to the Monge-Kantorovich Mass Transfer Problem. *Numerische Mathematik*, 84(3):375–393. DOI: 10.1007/s002110050002.

24. **Chen, S., Chewi, S., Lee, H., Li, Y., Lu, J., & Salim, A.** (2023). The Probability Flow ODE is Provably Fast. *NeurIPS 2023*. arXiv:2305.11798.

25. **Chung, H., Kim, J., McCann, M. T., Klasky, M. L., & Ye, J. C.** (2023). Diffusion Posterior Sampling for General Noisy Inverse Problems. *ICLR 2023* (Spotlight). arXiv:2209.14687.

26. **Feldman, S., Bates, S., & Romano, Y.** (2023). Calibrated Multiple-Output Quantile Regression with Representation Learning. *JMLR*, 24(24):1–48. arXiv:2110.00816.

27. **Hang, T., Gu, S., Li, C., Bao, J., Chen, D., Hu, H., Geng, X., & Guo, B.** (2023). Efficient Diffusion Training via Min-SNR Weighting Strategy. *ICCV 2023*, pp. 7441–7451. arXiv:2303.09556.

28. **Huang, J., Yang, G., Wang, Z., & Park, J. J.** (2024). DiffusionPDE: Generative PDE-Solving Under Partial Observation. *NeurIPS 2024*. arXiv:2406.17763.

29. **Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J., & Wasserman, L.** (2018). Distribution-Free Predictive Inference for Regression. *JASA*, 113(523):1094–1111. arXiv:1604.04173.

30. **Lin, S., Liu, B., Li, J., & Yang, X.** (2024). Common Diffusion Noise Schedules and Sample Steps are Flawed. *WACV 2024*, pp. 5404–5411. arXiv:2305.08891.

31. **Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M.** (2023). Flow Matching for Generative Modeling. *ICLR 2023*. arXiv:2210.02747.

32. **Ma, Z., Pitt, D., Azizzadenesheli, K., & Anandkumar, A.** (2024). Calibrated Uncertainty Quantification for Operator Learning via Conformal Prediction. *TMLR 2024*. arXiv:2402.01960.

33. **Salimans, T. & Ho, J.** (2022). Progressive Distillation for Fast Sampling of Diffusion Models. *ICLR 2022*. arXiv:2202.00512.

34. **Tong, A., Fatras, K., Malkin, N., Huguet, G., Zhang, Y., Rector-Brooks, J., Wolf, G., & Bengio, Y.** (2024). Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport. *TMLR 2024*. arXiv:2302.00482.

35. **Vovk, V., Gammerman, A., & Shafer, G.** (2005). *Algorithmic Learning in a Random World*. Springer. ISBN: 978-0-387-00152-4. DOI: 10.1007/b106715.

36. **Kossaifi, J., Kovachki, N., Li, Z., Pitt, D., Liu-Schiaffini, M., George, R. J., Bonev, B., Azizzadenesheli, K., Berner, J., Duruisseaux, V., & Anandkumar, A.** (2024). A Library for Learning Neural Operators. arXiv:2412.10354.

37. **Xu, X. & Chi, Y.** (2024). Provably Robust Score-Based Diffusion Posterior Sampling for Plug-and-Play Image Reconstruction. *NeurIPS 2024*. arXiv:2403.17042.

This document provides the theoretical backbone for the `laplace-uq-bench` *repository. Every equation maps to a specific source file (§13), every claim has a validation test (§14), and every approximation is marked explicitly. Numerical quantities are distinguished carefully: truncation error vs analytical solution, discrete residual of the linear system, and automated test tolerances are three separate things (§2.2). All posterior claims are scoped to the synthetic BC prior (§5.2). The project compares neural surrogates under controlled conditions — the theory ensures we know what we are measuring and why.*

Replication scope. All empirical results cited in this document (§§3.2, 7.5, 8, 9.3, 11.3, 11.5) are from **single training runs** — one checkpoint per model, evaluated on fixed test splits. No results are averaged over multiple seeds or replicated runs. Model rankings (e.g., "DDPM achieved the lowest functional CRPS") reflect this single-run comparison and should be interpreted accordingly: they demonstrate that the ranking is achievable, not that it is statistically guaranteed to hold under retraining. Replication across seeds is a planned follow-up.